

Aparcamiento Autónomo

Evaluación comparativa de controladores de seguimiento y Deep
Reinforcement Learning en un entorno de simulación
MATLAB/Simulink–Unreal Engine

José Antonio García Campanario
Andrés Martínez Márquez
Fernando Román Hidalgo

8 de junio de 2026

Índice general

1. Introducción	1
1.1. Motivación y contexto	1
1.2. Objetivos del proyecto	1
1.3. Estado del arte en aparcamiento autónomo	2
1.3.1. Planificadores geométricos clásicos: Reeds-Shepp y curvas de Dubins	2
1.3.2. Hybrid A*	2
1.3.3. Rapid-exploring Random Trees: RRT y RRT*	3
1.3.4. MPC puro para aparcamiento	3
1.3.5. Posicionamiento de la arquitectura propuesta	4
1.4. Alcance del documento	4
1.5. Estructura de la memoria	4
2. Marco teórico y fundamentos matemáticos	6
2.1. Modelado del vehículo	6
2.1.1. Variables de estado y entradas de control	6
2.1.2. Justificación del modelo cinemático frente al modelo dinámico . . .	6
2.1.3. Modelo cinemático continuo	9
2.1.4. Restricciones físicas	9
2.1.5. Linealización y discretización del modelo	9
2.2. Controladores de seguimiento de trayectorias	11
2.2.1. Pure Pursuit	11
2.2.2. Controlador Stanley	11
2.2.3. Linear Quadratic Regulator (LQR)	12
2.2.4. Adaptive Model Predictive Control	15
2.2.5. Nonlinear Model Predictive Control	17
2.3. Deep Reinforcement Learning (DRL)	18
2.3.1. Procesos de Decisión de Markov (MDP)	18
2.3.2. Función de valor, política y ecuación de Bellman	19
2.3.3. Agentes de control continuo utilizados	20
2.3.4. Interpretabilidad y limitaciones del DRL en conducción autónoma .	21
3. Metodología y entorno de simulación	23
3.1. Descripción general del entorno	23
3.2. Arquitectura del sistema propuesto	23

3.3.	Sensor Lidar 3D	23
3.4.	Planificación de trayectorias y referencia	24
3.5.	Diseño del entorno de aprendizaje DRL	24
3.5.1.	Espacio de observaciones y acciones	24
3.5.2.	Gestión de la velocidad longitudinal: módulo Stateflow	24
3.5.3.	Función de recompensa	25
3.5.4.	Criterios de finalización	26
4.	Fase 1: Aproximación al punto de aparcamiento	27
4.1.	Configuración de los experimentos de seguimiento	27
4.2.	Resultados cuantitativos de seguimiento	27
4.3.	Análisis individual de controladores	28
4.3.1.	Pure Pursuit	28
4.3.2.	Stanley	29
4.3.3.	LQR	31
4.3.4.	MPC adaptativo	32
4.3.5.	NMPC	33
4.4.	Comparativa de coste computacional	33
4.5.	Análisis conjunto: precisión frente a coste	34
4.6.	Discusión crítica: por qué MPC adaptativo iguala al NMPC	34
4.7.	Análisis de robustez ante incertidumbres	35
4.7.1.	Fuentes de incertidumbre	35
4.7.2.	Métricas estadísticas de robustez	36
5.	Fase 2: Maniobra de aparcamiento mediante DRL	37
5.1.	Configuración del entrenamiento	37
5.1.1.	Hiperparámetros de los algoritmos	37
5.1.2.	Arquitectura de redes neuronales	37
5.1.3.	Coste computacional de entrenamiento	38
5.1.4.	Viabilidad en hardware embarcado	38
5.2.	Dinámica de entrenamiento y convergencia	39
5.2.1.	Evolución de la recompensa acumulada por episodio	39
5.2.2.	Análisis de exploración frente a explotación	40
5.3.	Validación comparativa de los agentes DRL	41
5.3.1.	Configuración de los ensayos	41
5.3.2.	Resultados cuantitativos	41
5.3.3.	Análisis de los resultados	43
5.3.4.	Limitaciones estadísticas de la validación	44
5.3.5.	Perspectiva de optimización futura	45
6.	Conclusiones y trabajo futuro	46
6.1.	Conclusiones	46
6.2.	Aportaciones originales	46
6.3.	Discusión: control clásico frente a DRL	47

6.4.	Líneas de trabajo futuro	47
6.5.	Análisis de la brecha Sim-to-Real	48
6.5.1.	Ruido y limitaciones reales de la percepción Lidar	48
6.5.2.	Domain Randomization	49
6.5.3.	Errores de calibración del sensor	49
6.5.4.	Latencia computacional en hardware embarcado y multitarea	50
6.5.5.	Desfase dinámico: limitaciones del modelo cinemático y modelo dinámico lateral	51

Capítulo 1

Introducción

1.1. Motivación y contexto

El aparcamiento autónomo constituye una maniobra especialmente exigente dentro de la conducción autónoma. A diferencia de tareas de seguimiento de carril a velocidad media o alta, el aparcamiento requiere operar a baja velocidad, en un espacio geoméricamente restringido, con obstáculos próximos y con tolerancias finales reducidas en posición y orientación. Además, el vehículo está sujeto a restricciones no holonómicas: no puede desplazarse lateralmente de forma instantánea, por lo que la orientación y el radio de giro condicionan directamente la factibilidad de la maniobra.

El proyecto se plantea con una arquitectura híbrida. En primer lugar, el vehículo realiza una fase de aproximación o búsqueda siguiendo una trayectoria de referencia dentro del aparcamiento. Esta fase se aborda mediante controladores deterministas de seguimiento de trayectorias: *Pure Pursuit*, *Stanley Controller*, *Linear Quadratic Regulator* (LQR), *Adaptive Model Predictive Control* (Adaptive MPC) y *Nonlinear Model Predictive Control* (NMPC). Posteriormente, cuando el sistema detecta una plaza libre y alcanza la zona de transición, se activa un controlador basado en *Deep Reinforcement Learning* (DRL) encargado de ejecutar la maniobra final de estacionamiento.

La motivación técnica de esta división es clara. Los controladores clásicos y óptimos proporcionan trazabilidad matemática, bajo coste computacional y comportamiento predecible durante la aproximación. En cambio, la maniobra final de aparcamiento puede beneficiarse de una política aprendida, capaz de capturar estrategias de actuación no triviales en un entorno con restricciones geométricas severas y percepción simulada mediante sensores 3D.

1.2. Objetivos del proyecto

El objetivo general es diseñar, implementar y analizar una arquitectura de aparcamiento autónomo que combine control de seguimiento de trayectorias para la fase de aproximación y DRL para la fase final de estacionamiento.

Los objetivos específicos son:

- Definir el modelo cinemático utilizado por los controladores, identificando estados,

entradas, restricciones físicas y discretización.

- Implementar y documentar cinco controladores de aproximación: *Pure Pursuit*, *Stanley*, LQR, Adaptive MPC y NMPC.
- Integrar dichos controladores en el flujo de simulación MATLAB/Simulink, manteniendo una interfaz común basada en la pose del vehículo, la trayectoria de referencia y la acción de control longitudinal–lateral.
- Implementar y comparar tres agentes DRL (DDPG, TD3 y SAC) para la maniobra final de aparcamiento, evaluando precisión, tasa de éxito, suavidad de control y coste computacional.
- Evaluar la arquitectura completa sobre el mismo escenario y modelo de vehículo, para extraer conclusiones sobre el mejor equilibrio precisión–coste en cada fase.

1.3. Estado del arte en aparcamiento autónomo

El aparcamiento autónomo ha sido abordado desde múltiples perspectivas metodológicas en la literatura científica. A continuación se presentan los enfoques más relevantes, estableciendo el marco comparativo en el que se sitúa la arquitectura propuesta en este trabajo.

1.3.1. Planificadores geométricos clásicos: Reeds-Shepp y curvas de Dubins

Los caminos de Reeds-Shepp [9] son curvas óptimas de longitud mínima para vehículos con movimiento hacia adelante y hacia atrás, modelados como sistemas no holonómicos con radio de giro mínimo $R_{min} = L / \tan(\delta_{max})$. Están compuestos por segmentos de tres tipos: arcos de giro máximo hacia la izquierda (L), arcos de giro máximo hacia la derecha (R) y segmentos rectos (S), agrupados en 48 familias de secuencias de hasta tres elementos. Dado que el espacio de configuraciones del vehículo es $SE(2) = \mathbb{R}^2 \times S^1$, el camino de Reeds-Shepp resuelve el problema de planificación punto a punto con longitud mínima en tiempo $\mathcal{O}(1)$.

Su principal limitación para el aparcamiento en entornos reales radica en que son trayectorias de longitud mínima en ausencia total de obstáculos. Al introducirse restricciones geométricas —vehículos estacionados, columnas, bordillos— el camino original puede volverse infactible, y cualquier replanificación requiere aplicar deflaciones y extensiones que rompen la optimalidad. Las curvas de Dubins [10], por su parte, son la variante unidireccional (sin marcha atrás), que reduce el espacio de soluciones a seis familias pero sacrifica la capacidad de ejecutar maniobras de entrada en plazas perpendiculares.

1.3.2. Hybrid A*

El algoritmo Hybrid A* [11] extiende el planificador A* clásico al espacio continuo de poses (X, Y, θ) . En lugar de expandir nodos en una rejilla discreta de posiciones, expande nodos en el espacio tridimensional de configuraciones aplicando primitivas de movimiento

cinemáticamente factibles: arcos de radio fijo hacia adelante y hacia atrás. La función heurística combina dos términos: la distancia al objetivo en el espacio libre de obstáculos (obteniéndose mediante una búsqueda Dijkstra bidimensional) y la longitud del camino de Reeds-Shepp sin obstáculos hasta el objetivo. Esta heurística hace que Hybrid A* sea admisible y sea computacionalmente viable en aparcamientos de tamaño moderado.

La principal desventaja respecto a la arquitectura propuesta en este TFG es que Hybrid A* requiere un mapa métrico denso y preciso del entorno. Cualquier imprecisión en el mapa —derivada de errores de localización o de obstáculos dinámicos no cartografiados— convierte el camino planificado en infactible en ejecución, requiriendo replanificación en tiempo real. En contraste, el agente DRL propuesto actúa reactivamente sobre la percepción Lidar instantánea, adaptando la dirección en tiempo real sin necesidad de un mapa global.

1.3.3. Rapid-exploring Random Trees: RRT y RRT*

Los algoritmos RRT [12] y su variante asintóticamente óptima RRT* [13] son planificadores de muestreo aleatorio que construyen un árbol de exploración en el espacio de configuraciones. En cada iteración, se muestrea aleatoriamente una configuración $\mathbf{x}_{rand} \in \mathcal{C}_{free}$, se encuentra el nodo más cercano del árbol $\mathbf{x}_{near}$, y se extiende el árbol desde $\mathbf{x}_{near}$ hacia $\mathbf{x}_{rand}$ aplicando una primitiva de movimiento cinemáticamente factible. RRT* añade un paso de reconexión local (*rewiring*) que garantiza convergencia asintótica hacia el camino óptimo.

En el contexto del aparcamiento, RRT* proporciona mayor flexibilidad que Hybrid A* en entornos no estructurados, pero su convergencia puede ser lenta cuando el espacio libre tiene morfología estrecha —como los pasillos entre vehículos estacionados— y cuando se exigen tolerancias de pose muy ajustadas en el estado final (típicamente $\pm 0,1\text{ m}$ y $\pm 5^\circ$). Además, al igual que Hybrid A*, requiere representación explícita del entorno y sufre degradación de rendimiento ante obstáculos dinámicos o errores de localización acumulados.

1.3.4. MPC puro para aparcamiento

Varios trabajos proponen resolver la maniobra de aparcamiento completa mediante un único controlador MPC no lineal [14], eliminando la necesidad de un planificador de trayectorias separado. En esta formulación, el estado deseado $\mathbf{x}_{ref}$ es directamente la pose de la plaza de aparcamiento, y el MPC maximiza un funcional de coste sobre un horizonte largo ($N_p \geq 20$) que penaliza la distancia al objetivo y el esfuerzo de control.

Esta aproximación es elegante matemáticamente, pero presenta limitaciones prácticas críticas para aparcamientos en entornos reales. En primer lugar, el horizonte de predicción necesario para resolver la maniobra completa desde la entrada del aparcamiento es de varios centenares de pasos, lo que hace el problema computacionalmente intratable en tiempo real con modelos no lineales. En segundo lugar, la presencia de obstáculos requiere incorporar restricciones de evitación no convexas en el NLP, cuya resolución con garantías de factibilidad en tiempo real es un problema abierto.

1.3.5. Posicionamiento de la arquitectura propuesta

La Tabla 1.1 resume la comparativa de la arquitectura propuesta frente a los enfoques del estado del arte.

Tabla 1.1: Comparativa de la arquitectura propuesta con métodos del estado del arte.

Método	Mapa requeri- do	Obstáculos dinámi- cos	Coste compu- tacional	Tolerancia localiza- ción
Reeds-Shepp	No	No	Muy bajo	Alta
Hybrid A*	Sí (denso)	Limitado	Medio	Baja
RRT*	Sí (denso)	Limitado	Alto	Baja
MPC puro	Parcial	No	Muy alto	Media
Propuesto (Control Clásico+DRL)	Parcial	Sí (Lidar)	Medio	Media– Alta

La arquitectura híbrida propuesta combina la eficiencia en la fase de aproximación—donde la trayectoria de referencia está bien definida— con la capacidad reactiva y adaptativa del agente DRL para la fase de estacionamiento, donde la variabilidad de la configuración inicial y la presencia de obstáculos próximos hace inviable una planificación geométrica exhaustiva en tiempo real.

1.4. Alcance del documento

Este documento recoge de forma rigurosa la parte correspondiente al modelado del vehículo, a los controladores de seguimiento empleados en la fase de aproximación y al agente DRL utilizado en la maniobra final. Los resultados experimentales de seguimiento y de validación del agente se presentan con métricas cuantitativas homogéneas obtenidas en el entorno MATLAB/Simulink–Unreal Engine.

1.5. Estructura de la memoria

La memoria se organiza de la siguiente forma:

- El Capítulo 2 presenta el modelo cinemático del vehículo y la formulación matemática de todos los controladores, incluyendo la base teórica del DRL.
- El Capítulo 3 describe la metodología de simulación, la arquitectura de control y la interacción entre planificación, seguimiento, percepción simulada y conmutación hacia DRL.

- El Capítulo 4 presenta los resultados experimentales de la fase de aproximación con métricas cuantitativas comparativas.
- El Capítulo 5 recoge la maniobra de aparcamiento mediante DRL: configuración del entrenamiento, resultados y comparativa entre agentes.
- El Capítulo 6 recoge conclusiones y líneas de trabajo futuro.

Capítulo 2

Marco teórico y fundamentos matemáticos

2.1. Modelado del vehículo

2.1.1. Variables de estado y entradas de control

El vehículo se modela mediante un modelo cinemático básico. Este modelo agrupa las dos ruedas delanteras en una única rueda equivalente y las dos ruedas traseras en otra rueda equivalente, separadas por la batalla del vehículo. En los archivos del proyecto se emplea:

$$L = 2,8 \text{ m}, \quad (2.1)$$

donde L es la distancia entre ejes.

El vector de estado se define como:

$$\mathbf{x} = \begin{bmatrix} X & Y & \theta \end{bmatrix}^T, \quad (2.2)$$

donde X e Y son las coordenadas globales del vehículo en el plano y θ es el ángulo de guiñada o *yaw*. El vector de entrada es:

$$\mathbf{u} = \begin{bmatrix} v & \delta \end{bmatrix}^T, \quad (2.3)$$

donde v es la velocidad longitudinal comandada y δ es el ángulo de dirección. Aunque algunos bloques de Simulink nombran la señal de salida como `accel_steer`, en el modelo cinemático utilizado por los controladores la primera componente se interpreta como velocidad longitudinal v , no como aceleración.

2.1.2. Justificación del modelo cinemático frente al modelo dinámico

La elección del modelo cinemático empleado frente a un modelo dinámico completo requiere una justificación rigurosa, ya que condiciona directamente la validez de todos los

controladores implementados. A continuación se expone el fundamento físico y matemático que avala esta elección en el contexto del aparcamiento a baja velocidad.

Condición de rodadura pura y número de Stribeck

El modelo cinemático asume que las ruedas ruedan sin deslizamiento lateral, es decir, que el vector velocidad en el punto de contacto de cada neumático es paralelo al plano de dicho neumático. Esta condición se denomina *rodadura pura* o *no deslizamiento lateral*. En términos del ángulo de deriva del neumático α , la condición equivale a $\alpha \approx 0$.

La fuerza lateral que desarrolla un neumático se puede modelar, en la región lineal, mediante la rigidez de deriva C_α como:

$$F_y = -C_\alpha \alpha. \quad (2.4)$$

El ángulo de deriva del neumático trasero se puede aproximar como:

$$\alpha_r \approx \frac{v_y - b\dot{\theta}}{v_x}, \quad (2.5)$$

donde v_y es la velocidad lateral del centro de gravedad, b es la distancia del CG al eje trasero y $\dot{\theta}$ es la velocidad de guiñada. La aceleración lateral que produce la fuerza lateral sobre la masa del vehículo es $a_y = F_y/m$. Aplicando la segunda ley de Newton en la dirección lateral del chasis:

$$m a_y = m (\dot{v}_y + v_x \dot{\theta}) = F_{yf} + F_{yr}. \quad (2.6)$$

Para que la hipótesis cinemática sea válida ($\alpha \approx 0$), es necesario que la aceleración lateral centrípeta sea despreciable frente a la gravedad. El criterio habitual en la bibliografía de vehículos autónomos [2] fija la condición:

$$\frac{a_y}{g} = \frac{v^2/R}{g} \ll 1, \quad (2.7)$$

donde $R = L/\tan \delta$ es el radio de curvatura instantáneo. Para las velocidades empleadas en el aparcamiento ($v \leq 5$ m/s) y el radio mínimo de giro del vehículo ($R_{min} = L/\tan(\pi/4) = 2,8$ m), la aceleración lateral máxima es:

$$a_{y,max} = \frac{v_{max}^2}{R_{min}} = \frac{25}{2,8} \approx 8,93 \text{ m/s}^2. \quad (2.8)$$

Sin embargo, en condiciones normales de aparcamiento ($v \leq 2,5$ m/s y $R \geq 5$ m):

$$a_{y,typ} = \frac{(2,5)^2}{5} = 1,25 \text{ m/s}^2, \quad \frac{a_{y,typ}}{g} \approx 0,13, \quad (2.9)$$

que es suficientemente pequeño para que el ángulo de deriva permanezca en la región lineal del neumático ($|\alpha| < 5^\circ$ para neumáticos de turismo convencionales). Esto justifica

que la hipótesis de rodadura pura sea válida en la gran mayoría de las maniobras de aparcamiento.

Velocidad crítica de transición cinemática–dinámica

Se puede definir matemáticamente una velocidad crítica v_c por debajo de la cual el modelo cinemático es suficientemente preciso. A partir de la ecuación del movimiento lateral linealizado del modelo dinámico [2], la respuesta en frecuencia del ángulo de deriva del eje trasero α_r a una entrada de ángulo de dirección δ presenta un polo en:

$$s_1 = -\frac{C_{\alpha f} + C_{\alpha r}}{mv}, \quad (2.10)$$

cuya frecuencia natural decrece con la velocidad. Para $v \rightarrow 0$, $s_1 \rightarrow -\infty$, lo que implica que la dinámica lateral colapsa a una respuesta instantánea: el sistema se comporta como puramente cinemático. La velocidad crítica de transición se define como el valor de v para el cual el tiempo de constante del polo lateral es comparable al período de muestreo T_s :

$$\tau_1 = \frac{mv}{C_{\alpha f} + C_{\alpha r}} \lesssim T_s = 0,1 \text{ s}. \quad (2.11)$$

Para valores típicos de un turismo mediano ($m = 1500 \text{ kg}$, $C_{\alpha f} = C_{\alpha r} = 50\,000 \text{ N/rad}$):

$$v_c = \frac{T_s(C_{\alpha f} + C_{\alpha r})}{m} = \frac{0,1 \times 100\,000}{1500} \approx 6,67 \text{ m/s}. \quad (2.12)$$

Puesto que el aparcamiento opera con $v \leq 5 \text{ m/s} < v_c$, se confirma cuantitativamente que la dinámica lateral es significativamente más rápida que el período de muestreo, y por tanto el modelo cinemático constituye una aproximación válida para el diseño de los controladores.

Resumen de la justificación

La elección del modelo cinemático está respaldada por tres argumentos complementarios: (i) el número adimensional de aceleración lateral típico ($a_y/g \approx 0,13$) es suficientemente pequeño para mantener los ángulos de deriva dentro de la región lineal del neumático; (ii) la velocidad de operación ($v \leq 5 \text{ m/s}$) es inferior a la velocidad crítica de transición cinemática–dinámica ($v_c \approx 6,67 \text{ m/s}$); y (iii) los efectos de transferencia de carga y deslizamiento longitudinal son despreciables a las aceleraciones desarrolladas durante el aparcamiento. La incorporación de un modelo dinámico completo añadiría un mayor coste computacional y una complejidad de parametrización significativa —identificación de $C_{\alpha f}$, $C_{\alpha r}$, m , I_z — sin aportar mejoras observables en la precisión del seguimiento bajo las condiciones de operación del aparcamiento.

2.1.3. Modelo cinemático continuo

La dinámica continua implementada en `vehicleStateFcn.m` es:

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \frac{v}{L} \tan \delta \end{bmatrix}. \quad (2.13)$$

Este modelo es adecuado para maniobras de baja velocidad en las que se desprecia la dinámica lateral de neumáticos, transferencia de carga y deslizamiento. La hipótesis fundamental es que el vehículo rueda sin deriva lateral significativa, por lo que la curvatura instantánea viene dada por:

$$\kappa = \frac{\tan \delta}{L}, \quad R = \frac{1}{\kappa} = \frac{L}{\tan \delta}, \quad (2.14)$$

siendo R el radio instantáneo de giro.

2.1.4. Restricciones físicas

Las restricciones empleadas de forma común en los controladores son:

$$-5 \text{ m/s} \leq v \leq 5 \text{ m/s}, \quad -\frac{\pi}{4} \leq \delta \leq \frac{\pi}{4}. \quad (2.15)$$

En el controlador *Pure Pursuit* implementado se utiliza una saturación más conservadora para la dirección:

$$-0,6 \leq \delta \leq 0,6 \quad \text{rad}, \quad (2.16)$$

con el objetivo de reducir oscilaciones en seguimiento geométrico.

2.1.5. Linealización y discretización del modelo

Los controladores basados en modelos lineales, especialmente LQR y Adaptive MPC, requieren disponer de una aproximación lineal local de (2.13). La linealización continua alrededor de un punto de operación $(\mathbf{x}_0, \mathbf{u}_0)$ se obtiene mediante los Jacobianos:

$$A = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}_0, \mathbf{u}_0}, \quad B = \left. \frac{\partial f}{\partial \mathbf{u}} \right|_{\mathbf{x}_0, \mathbf{u}_0}. \quad (2.17)$$

Para el modelo elegido:

$$A = \begin{bmatrix} 0 & 0 & -v \sin \theta \\ 0 & 0 & v \cos \theta \\ 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} \cos \theta & 0 \\ \sin \theta & 0 \\ \frac{1}{L} \tan \delta & \frac{v}{L} \sec^2 \delta \end{bmatrix}. \quad (2.18)$$

La salida se toma como el propio estado, por lo que $C = I_3$ y $D = 0_{3 \times 2}$.

La linealización del sistema continuo (2.13) en torno a la pose y entrada nominales $(\mathbf{x}_0, \mathbf{u}_0)$ conduce al sistema afín continuo:

$$\dot{\mathbf{x}}(t) \approx f(\mathbf{x}_0, \mathbf{u}_0) + A(\mathbf{x}(t) - \mathbf{x}_0) + B(\mathbf{u}(t) - \mathbf{u}_0) = A\mathbf{x}(t) + B\mathbf{u}(t) + \mathbf{d}_c, \quad (2.19)$$

donde el término residual continuo $\mathbf{d}_c$ se define como:

$$\mathbf{d}_c = f(\mathbf{x}_0, \mathbf{u}_0) - A\mathbf{x}_0 - B\mathbf{u}_0. \quad (2.20)$$

Para su implementación digital en los controladores predictivos y óptimos, este sistema continuo afín debe ser discretizado con un tiempo de muestreo T_s . Si se emplea una discretización por diferencias finitas hacia adelante de Euler (aproximación de primer orden):

$$\frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{T_s} \approx A\mathbf{x}_k + B\mathbf{u}_k + \mathbf{d}_c, \quad (2.21)$$

lo que resulta directamente en el modelo afín discretizado:

$$\mathbf{x}_{k+1} \approx A_d\mathbf{x}_k + B_d\mathbf{u}_k + \mathbf{d}_x, \quad (2.22)$$

donde las matrices del sistema discretizado y el término constante $\mathbf{d}_x$ vienen dados por:

$$A_d = I_3 + T_s A, \quad B_d = T_s B, \quad \mathbf{d}_x = T_s \mathbf{d}_c = T_s (f(\mathbf{x}_0, \mathbf{u}_0) - A\mathbf{x}_0 - B\mathbf{u}_0). \quad (2.23)$$

Alternativamente, en sistemas con dinámicas rápidas o para mayor precisión, la función `vehicleStateJacobianFcnDT.m` realiza la discretización exacta asumiendo retención de orden cero (ZOH) en la señal de control. Bajo esta hipótesis, las matrices discretas afines y la señal constante $\mathbf{d}_x$ se calculan mediante la exponencial de matriz del sistema ampliado:

$$A_d = e^{AT_s}, \quad B_d = \left(\int_0^{T_s} e^{A\tau} d\tau \right) B, \quad \mathbf{d}_x = \left(\int_0^{T_s} e^{A\tau} d\tau \right) \mathbf{d}_c. \quad (2.24)$$

Cuando la matriz A es invertible, las integrales anteriores admiten soluciones analíticas directas:

$$B_d = A^{-1}(A_d - I_3)B, \quad \mathbf{d}_x = A^{-1}(A_d - I_3)\mathbf{d}_c. \quad (2.25)$$

No obstante, en el modelo cinemático (2.13), la matriz A definida en (2.18) es singular (rango menor que 3), por lo que las integrales anteriores se evalúan numéricamente mediante aproximaciones de Taylor de la exponencial de matriz ampliada o en series de potencias.

En el bloque Adaptive MPC de Simulink, este término de desfase se proporciona en la variable **DX** (que corresponde exactamente a $\mathbf{d}_x$), alimentando al optimizador junto con las matrices lineales instantáneas **A** (A_d), **B** (B_d), **C** ($C_d = I_3$), **D** ($D_d = 0_{3 \times 2}$) y los puntos de operación nominales **X** ($\mathbf{x}_0$), **U** ($\mathbf{u}_0$) e **Y** ($\mathbf{y}_0$).

2.2. Controladores de seguimiento de trayectorias

2.2.1. Pure Pursuit

Principio de funcionamiento

Pure Pursuit es un controlador geométrico. Su objetivo es seleccionar un punto objetivo sobre la trayectoria, situado a una distancia de anticipación L_d , y calcular el ángulo de dirección que haría que el vehículo describiera un arco circular hacia dicho punto.

La implementación utilizada desplaza primero el punto de cálculo desde la pose del vehículo hacia el eje trasero:

$$X_r = X - \frac{L}{2} \cos \theta, \quad Y_r = Y - \frac{L}{2} \sin \theta. \quad (2.26)$$

Distancia de anticipación adaptativa

En el código se estima una medida discreta de curvatura comparando la orientación de la trayectoria en el punto cercano y en un punto situado 15 muestras por delante:

$$\Delta\theta_{\text{path}} = \text{wrap}_{[-\pi, \pi]} (\theta_{\text{ref}}(i + 15) - \theta_{\text{ref}}(i)), \quad \kappa_{\text{norm}} = \min \left(1, \frac{|\Delta\theta_{\text{path}}|}{0,3} \right). \quad (2.27)$$

La distancia de anticipación se interpola linealmente:

$$L_d = L_{d,\text{recta}} - \kappa_{\text{norm}} (L_{d,\text{recta}} - L_{d,\text{curva}}), \quad L_{d,\text{recta}} = 20 \text{ m}, \quad L_{d,\text{curva}} = 6 \text{ m}. \quad (2.28)$$

Ley de control

Dado el punto objetivo (X_t, Y_t) , el ángulo relativo en el marco del vehículo se calcula como:

$$\alpha = \text{wrap}_{[-\pi, \pi]} (\text{atan2}(Y_t - Y_r, X_t - X_r) - \theta), \quad \delta = \text{atan2}(2L \sin \alpha, L_d). \quad (2.29)$$

Se añade una amortiguación progresiva cuando $|\alpha| < 0,1$ rad:

$$s(\alpha) = \max \left(0, 2, \left(\frac{|\alpha|}{0,1} \right)^2 \right), \quad \delta \leftarrow s(\alpha)\delta, \quad \mathbf{u}_{PP} = [2, 0 \quad \text{sat}_{[-0,6,0,6]}(\delta)]^T. \quad (2.30)$$

2.2.2. Controlador Stanley

Principio de funcionamiento

El controlador Stanley combina un término de error de orientación con un término de error lateral medido en el eje delantero. En la implementación se define el punto de control delantero como:

$$X_f = X + d_f \cos \theta, \quad Y_f = Y + d_f \sin \theta, \quad d_f = 2,8 \text{ m}. \quad (2.31)$$

Justificación geométrica de $d_f = L$. En el sistema de simulación empleado, el vector de estado del vehículo $\mathbf{x} = [X, Y, \theta]^T$ se refiere al centro del eje trasero del vehículo, que constituye el origen del sistema de referencia del cuerpo rígido (**egoPose**). Esta convención es estándar en el modelo cinemático elegido y coincide con la implementación de `vehicleStateFcn.m`.

Bajo esta hipótesis, la posición del centro del eje delantero se obtiene desplazando el origen una distancia igual a la batalla L en la dirección del encabecamiento θ :

$$\begin{bmatrix} X_f \\ Y_f \end{bmatrix} = \begin{bmatrix} X \\ Y \end{bmatrix} + L \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}. \quad (2.32)$$

Esta expresión coincide exactamente con (2.31) al fijar $d_f = L = 2,8$ m. Por tanto, la implementación es correcta y coherente con la teoría del controlador Stanley, que define el error lateral respecto al eje delantero del vehículo [8]. Si la **egoPose** estuviera referida al centro geométrico del vehículo en lugar del eje trasero, el valor correcto sería $d_f = L/2$.

Error lateral y error de orientación

El error lateral firmado se calcula como:

$$e_{lat} = (X_f - X_c) \sin \theta_c - (Y_f - Y_c) \cos \theta_c, \quad (2.33)$$

donde (X_c, Y_c, θ_c) es el punto de la trayectoria más cercano al punto delantero. El error de orientación se calcula respecto a un punto futuro situado 30 muestras por delante para mejorar la anticipación en curva:

$$\theta_e = \text{wrap}_{[-\pi, \pi]} (\theta_{\text{ref}}(i + 30) - \theta). \quad (2.34)$$

Ley de control

$$\delta = \theta_e + \arctan \left(\frac{k_{st} e_{lat}}{v_{ref} + k_{soft}} \right), \quad v_{ref} = 2,5 \text{ m/s}, \quad k_{st} = 2,0, \quad k_{soft} = 1,0, \quad (2.35)$$

con saturación final $\mathbf{u}_{Stanley} = [2,5 \quad \text{sat}_{[-\pi/4, \pi/4]}(\delta)]^T$.

2.2.3. Linear Quadratic Regulator (LQR)

Análisis de controlabilidad

Antes de diseñar el regulador LQR es necesario verificar que el sistema linealizado discreto (A_d, B_d) es controlable, condición necesaria para que la DARE (2.41) tenga solución única definida positiva y la ganancia K_{lqr} estabilice el sistema en lazo cerrado.

La condición de controlabilidad del par (A_d, B_d) se verifica calculando la **matriz de**

controlabilidad de Kalman:

$$\mathcal{C} = \begin{bmatrix} B_d & A_d B_d & A_d^2 B_d \end{bmatrix} \in \mathbb{R}^{3 \times 6}, \quad (2.36)$$

y comprobando que $\text{rank}(\mathcal{C}) = 3$ (igual al orden del sistema). El par (A_d, B_d) es controlable si y solo si $\text{rank}(\mathcal{C}) = 3$.

Evaluando las matrices en el punto de operación nominal ($\theta_0 = 0$, $\delta_0 = 0$, $v_0 = 2,5 \text{ m/s}$, $T_s = 0,1 \text{ s}$):

$$A_d = I_3 + T_s A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & v_0 T_s \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0,25 \\ 0 & 0 & 1 \end{bmatrix}, \quad B_d = T_s B = \begin{bmatrix} 0,1 & 0 \\ 0 & 0 \\ \frac{T_s}{L} & \frac{v_0 T_s}{L} \end{bmatrix} \approx \begin{bmatrix} 0,1 & 0 \\ 0 & 0 \\ 0,0357 & 0,0893 \end{bmatrix}. \quad (2.37)$$

El rango de $\mathcal{C}$ puede verificarse numéricamente con MATLAB mediante `rank(ctrb(Ad, Bd))`. Para el modelo de seleccionado con movimiento hacia adelante ($v_0 \neq 0$), el rango es 3, confirmando la controlabilidad completa del sistema. Físicamente, esto implica que existen secuencias de entradas (v, δ) que llevan el vehículo desde cualquier pose inicial hasta cualquier pose final en tiempo finito, respetando las restricciones de movimiento no holonómico.

Interpretación física de la controlabilidad. El estado X se controla directamente por v a través de $B_d(1, 1) \neq 0$. Los estados Y y θ están acoplados: θ se controla directamente por δ a través de $B_d(3, 2) \neq 0$, y Y se controla indirectamente a través de la dinámica de θ (propagada en dos pasos: $\delta \rightarrow \theta \rightarrow Y$), lo que explica la aparición del término $A_d^2 B_d$ en (2.36) para el rango completo. Esta cadena de controlabilidad indirecta explica por qué el LQR asigna un peso mayor a e_{lat} (componente Y del error) que a e_θ en la matriz Q : el estado Y es el de mayor inercia de control en la cadena de propagación.

Modelo lineal de diseño y formulación matemática

El regulador cuadrático lineal en tiempo discreto (LQR) se diseña a partir del modelo lineal discretizado en torno al punto de operación nominal. El objetivo es minimizar un funcional de coste de horizonte infinito que penaliza de forma cuadrática tanto las desviaciones del vector de error de estado $\mathbf{e}_k$ como el esfuerzo del comando de control corrector $\mathbf{u}_{corr,k} = \mathbf{u}_k - \mathbf{u}_{nom}$:

$$J = \sum_{k=0}^{\infty} \left(\mathbf{e}_k^T Q \mathbf{e}_k + \mathbf{u}_{corr,k}^T R \mathbf{u}_{corr,k} \right), \quad (2.38)$$

donde las matrices de ponderación simétricas y semidefinida/definida positiva se seleccionan en el código como:

$$Q = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 15 & 0 \\ 0 & 0 & 5 \end{bmatrix}, \quad R = \begin{bmatrix} 0,1 & 0 \\ 0 & 5 \end{bmatrix}. \quad (2.39)$$

La matriz Q asigna una penalización significativamente mayor a la segunda componente del error (error lateral), forzando al controlador a concentrar el esfuerzo de guiado en mantener la proximidad a la trayectoria de referencia.

De acuerdo con la teoría de control óptimo en tiempo discreto, la ganancia óptima de realimentación de estado K_{lqr} que minimiza (2.38) viene dada por:

$$K_{lqr} = \left(R + B_d^T P B_d \right)^{-1} B_d^T P A_d, \quad (2.40)$$

donde $P \in \mathbb{R}^{3 \times 3}$ es la única matriz simétrica definida positiva que resuelve la Ecuación Algebraica de Riccati en Tiempo Discreto (DARE):

$$P = A_d^T P A_d - A_d^T P B_d \left(R + B_d^T P B_d \right)^{-1} B_d^T P A_d + Q. \quad (2.41)$$

La función de MATLAB `dlqr` resuelve numéricamente la ecuación (2.41) a partir de las matrices del sistema discretizado A_d, B_d y las ponderaciones Q, R , devolviendo el vector de ganancias constante.

Estabilidad del sistema en lazo cerrado

La ganancia LQR K_{lqr} garantiza que el sistema en lazo cerrado $A_{cl} = A_d - B_d K_{lqr}$ sea asintóticamente estable, siempre que el par (A_d, B_d) sea controlable y el par $(A_d, Q^{1/2})$ sea observable. En este caso, todos los valores propios de A_{cl} tienen módulo estrictamente menor que la unidad:

$$\rho(A_{cl}) = \max_i |\lambda_i(A_{cl})| < 1, \quad (2.42)$$

donde $\rho(\cdot)$ denota el radio espectral. Esta propiedad implica que el error de seguimiento $\mathbf{e}_k \rightarrow \mathbf{0}$ de forma exponencial a medida que $k \rightarrow \infty$ bajo perturbaciones nulas, lo que garantiza la estabilidad asintótica del sistema de control. Cabe destacar que esta estabilidad es local, válida en la vecindad del punto de linealización; la degradación de la aproximación lineal en curvatura elevada explica el aumento del error lateral en los tramos curvos reportado en la Sección 4.3.3.

Error en marco local y ley de control

El vector de estado de error $\mathbf{e}_k = [e_x, e_{lat}, e_\theta]^T$ representa las desviaciones del vehículo en el marco de coordenadas local intrínseco de la trayectoria de referencia, determinado por el punto de referencia más cercano (X_r, Y_r, θ_r) :

$$\mathbf{e}_k = \begin{bmatrix} e_{lon,k} \\ e_{lat,k} \\ e_{\theta,k} \end{bmatrix} = \begin{bmatrix} \cos \theta_r & \sin \theta_r & 0 \\ -\sin \theta_r & \cos \theta_r & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_k - X_r \\ Y_k - Y_r \\ \text{wrap}_{[-\pi, \pi]}(\theta_k - \theta_r) \end{bmatrix}, \quad (2.43)$$

donde:

- e_{lon} es el error longitudinal a lo largo del camino de referencia.
- e_{lat} es el error lateral o distancia perpendicular firmada desde el eje trasero del

vehículo a la trayectoria de referencia.

- e_θ es el error de orientación o desviación angular del encabezamiento del vehículo respecto a la tangente del camino de referencia.

Una vez proyectado el error instantáneo, la ley de control óptima calcula la acción correctiva lineal y la superpone sobre la acción nominal de avance $\mathbf{u}_{nom} = [2,5, 0]^T$:

$$\mathbf{u}_k = \mathbf{u}_{nom} - K_{lqr} \mathbf{e}_k. \quad (2.44)$$

2.2.4. Adaptive Model Predictive Control

Estabilidad recursiva y factibilidad recursiva

Uno de los aspectos más críticos del diseño de cualquier controlador MPC en aplicaciones reales es garantizar no solo que el problema de optimización tenga solución en $k = 0$, sino que sea **recursivamente factible**: que si el problema es factible en el instante k , también lo sea en $k + 1$. La falta de factibilidad recursiva puede dar lugar a situaciones en las que el optimizador no encuentre solución factible en tiempo real, dejando al sistema sin acción de control.

En el caso del Adaptive MPC con restricciones duras (*hard constraints*) sobre las entradas, la factibilidad recursiva se puede garantizar a través de varios mecanismos. El más utilizado en la práctica es la inclusión de un **conjunto terminal** $\mathcal{X}_f$ y una **restricción de estado terminal** $\mathbf{x}_{k+N_p|k} \in \mathcal{X}_f$, junto con una **función de coste terminal** $V_f(\mathbf{x}_{k+N_p|k})$ tal que:

$$\exists u \in \mathcal{U} : \mathbf{x}_{f,+} = A_d \mathbf{x}_f + B_d u + \mathbf{d}_x \in \mathcal{X}_f, \quad V_f(\mathbf{x}_{f,+}) \leq V_f(\mathbf{x}_f) - \mathbf{x}_f^T Q_y \mathbf{x}_f, \quad (2.45)$$

para todo $\mathbf{x}_f \in \mathcal{X}_f$. Bajo esta condición, la solución del paso k con el primer control aplicado y los restantes $N_p - 1$ controles del horizonte anterior más un control terminal auxiliar constituye una solución factible en $k + 1$.

En la implementación del Adaptive MPC de este trabajo, aunque no se impone explícitamente una restricción terminal, la función de coste con horizonte $N_p = 10$ y la actualización continua del modelo lineal actúan como mecanismo implícito de regularización, garantizando en la práctica la factibilidad en los escenarios de curvatura moderada del aparcamiento. Las restricciones blandas (*soft constraints*) sobre las salidas, que convierten las restricciones infactibles en penalizaciones adicionales en el coste, proporcionan una garantía más robusta para evitar la infactibilidad en escenarios extremos.

Desde el punto de vista de la **estabilidad**, el Adaptive MPC garantiza estabilidad asintótica del sistema en lazo cerrado si el conjunto terminal $\mathcal{X}_f$ es una región positivamente invariante para el sistema bajo la ley de control local y si la función de coste terminal V_f es una función de Lyapunov local. En ausencia de restricciones terminales explícitas, la estabilidad práctica (convergencia a un entorno acotado del punto de operación) se infiere del comportamiento empírico de las métricas de error, que muestran RMSE lateral de 0,0061 m y error máximo de 0,0203 m en los experimentos de seguimiento.

Configuración del controlador y vector de salidas

El control predictivo adaptativo (Adaptive MPC) discretiza y linealiza el modelo no lineal en cada paso de control k para formular un problema de optimización cuadrático con restricciones. En el archivo `createMPCForParking3D.m` se establecen los siguientes parámetros de diseño:

$$T_s = 0,1 \text{ s}, \quad N_p = 10, \quad N_c = 10, \quad (2.46)$$

donde N_p representa el horizonte de predicción y N_c el horizonte de control. Las salidas controladas del sistema se definen mediante la relación lineal $\mathbf{y}_{k+i|k} = C\mathbf{x}_{k+i|k}$, con $C = I_3$, lo que implica que el vector de salidas coincide con el vector de poses del vehículo $[X, Y, \theta]^T$. Las matrices de ponderación de la función de coste se definen de forma diagonal a partir de los pesos $W_y = [2, 2, 3]^T$ y $W_{\Delta u} = [0,1, 0,2]^T$:

$$Q_y = C^T \text{diag}(W_y)^2 C = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 9 \end{bmatrix}, \quad R_{\Delta u} = \text{diag}(W_{\Delta u})^2 = \begin{bmatrix} 0,01 & 0 \\ 0 & 0,04 \end{bmatrix}. \quad (2.47)$$

Formulación canónica del Problema de Programación Cuadrática (QP)

El vector de variables de decisión del optimizador en el instante k se define en términos de los incrementos de la acción de control a lo largo del horizonte de control N_c :

$$\Delta U_k = [\Delta \mathbf{u}_{k|k}^T \quad \Delta \mathbf{u}_{k+1|k}^T \quad \dots \quad \Delta \mathbf{u}_{k+N_c-1|k}^T]^T \in \mathbb{R}^{2N_c}, \quad (2.48)$$

donde $\Delta \mathbf{u}_{k+i|k} = \mathbf{u}_{k+i|k} - \mathbf{u}_{k+i-1|k}$. La acción de control física se recupera recursivamente como $\mathbf{u}_{k+i|k} = \mathbf{u}_{k+i-1|k} + \Delta \mathbf{u}_{k+i|k}$.

El problema de optimización se reformula en cada paso como un problema de programación cuadrática (QP) en su forma canónica:

$$\begin{aligned} \min_{\Delta U_k} \quad & \frac{1}{2} \Delta U_k^T H_k \Delta U_k + G_k^T \Delta U_k \\ \text{s.a.} \quad & M_k \Delta U_k \leq V_k, \end{aligned} \quad (2.49)$$

donde el Hessiano $H_k \in \mathbb{R}^{2N_c \times 2N_c}$ y el gradiente $G_k \in \mathbb{R}^{2N_c}$ se obtienen sustituyendo la predicción del estado afín discreto:

$$\mathbf{x}_{k+i+1|k} = A_{d,k} \mathbf{x}_{k+i|k} + B_{d,k} \mathbf{u}_{k+i|k} + \mathbf{d}_x \quad (2.50)$$

a lo largo de todo el horizonte de predicción. Las restricciones lineales $M_k \Delta U_k \leq V_k$ codifican tanto los límites físicos absolutos de la entrada como las restricciones de velocidad de cambio en las señales de control:

$$\mathbf{u}_{min} \leq \mathbf{u}_{k+i|k} \leq \mathbf{u}_{max}, \quad \Delta \mathbf{u}_{min} \leq \Delta \mathbf{u}_{k+i|k} \leq \Delta \mathbf{u}_{max}. \quad (2.51)$$

Una vez resuelto el QP, se aplica únicamente el primer incremento óptimo $\Delta \mathbf{u}_{k|k}^*$, recalculando el problema en el paso $k + 1$ bajo el enfoque de horizonte deslizante.

2.2.5. Nonlinear Model Predictive Control

Factibilidad recursiva en el NMPC

En el NMPC, la factibilidad recursiva es matemáticamente más compleja de garantizar que en el MPC lineal, debido a la no convexidad intrínseca del NLP resultante. El solucionador SQP puede quedar atrapado en mínimos locales o declarar infactibilidad numérica incluso cuando existe una solución factible para el NLP original. Para garantizar la factibilidad recursiva en la práctica, la implementación de NMPC empleada aplica dos mecanismos complementarios:

1. **Inicialización caliente** (*warm start*): el vector de variables de decisión Z_{k+1} se inicializa desplazando la solución óptima del instante anterior Z_k^* un paso hacia adelante:

$$Z_{k+1}^{init} = \begin{bmatrix} \mathbf{u}_{k+1|k}^{*T} & \cdots & \mathbf{u}_{k+N_c-1|k}^{*T} & \mathbf{u}_{k+N_c-1|k}^{*T} \end{bmatrix}^T, \quad (2.52)$$

replicando el último control óptimo como solución terminal. Esta inicialización explota la continuidad temporal de la solución óptima para reducir el número de iteraciones del SQP y acercar el punto de inicio al mínimo local de interés.

2. **Restricciones blandas de salida**: las restricciones de evasión de obstáculos se implementan como penalizaciones cuadráticas en la función de coste en lugar de restricciones duras, garantizando que el NLP siempre tenga un punto interior factible y evitando declaraciones de infactibilidad numérica por parte del solucionador.

Estructura matemática del problema de optimización

A diferencia de la formulación adaptativa, el NMPC utiliza directamente la dinámica continua no lineal del modelo (2.13) discretizada con un integrador numérico de alta fidelidad $f_d(\mathbf{x}, \mathbf{u})$ (típicamente Runge-Kutta de 4.º orden, RK4). Las variables de decisión se agrupan en el vector de entradas absolutas sobre el horizonte de control:

$$Z_k = \begin{bmatrix} \mathbf{u}_{k|k}^T & \mathbf{u}_{k+1|k}^T & \cdots & \mathbf{u}_{k+N_c-1|k}^T \end{bmatrix}^T \in \mathbb{R}^{2N_c}. \quad (2.53)$$

El problema de optimización resultante se formula como un problema de programación no lineal (NLP):

$$\begin{aligned} \min_{Z_k} \quad & J(Z_k, \mathbf{x}_k) = \sum_{i=1}^{N_p} \left\| \mathbf{x}_{k+i|k} - \mathbf{r}_{k+i|k} \right\|_{Q_y}^2 + \sum_{i=0}^{N_c-1} \left\| \Delta \mathbf{u}_{k+i|k} \right\|_{R_{\Delta u}}^2 + \sum_{i=0}^{N_c-1} \left\| \mathbf{u}_{k+i|k} \right\|_{R_u}^2 \\ \text{s.a.} \quad & \mathbf{x}_{k+i+1|k} = f_d(\mathbf{x}_{k+i|k}, \mathbf{u}_{k+i|k}), \\ & \mathbf{u}_{min} \leq \mathbf{u}_{k+i|k} \leq \mathbf{u}_{max}, \\ & \Delta \mathbf{u}_{min} \leq \Delta \mathbf{u}_{k+i|k} \leq \Delta \mathbf{u}_{max}. \end{aligned} \quad (2.54)$$

El Hessiano y el gradiente de la función de coste no lineal $J(Z_k, \mathbf{x}_k)$ respecto a las variables

de decisión no son constantes, por lo que el NLP se resuelve numéricamente mediante métodos iterativos de Programación Cuadrática Secuencial (SQP) o algoritmos de punto interior en cada instante de muestreo. Esto explica su elevado tiempo de cómputo por paso ($\approx 4,222$ ms), aunque proporciona mayor fidelidad ante ángulos de dirección extremos donde las no linealidades trigonométricas de la guiñada son críticas.

Para obtener los parámetros necesarios del NMPC, se ha de ejecutar el archivo `createNonLinearMPCForParking3D.m`

2.3. Deep Reinforcement Learning (DRL)

2.3.1. Procesos de Decisión de Markov (MDP)

La maniobra de aparcamiento se formula como un Proceso de Decisión de Markov (MDP), definido por la tupla:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma). \quad (2.55)$$

- **Espacio de estados $\mathcal{S}$** (o espacio de observaciones): el vector de estado $\mathbf{s}_t$ se compone de los errores de pose localizados respecto al punto de estacionamiento y de la nube de puntos Lidar procesada $\mathcal{P}_t$:

$$\mathbf{s}_t = (X_{e,n}, Y_{e,n}, \theta_{e,n}, \mathcal{P}_{t,n}), \quad (2.56)$$

donde los errores físicos de posición y orientación se normalizan según:

$$X_{e,n} = \frac{X_{obj} - X}{D_{max}}, \quad Y_{e,n} = \frac{Y_{obj} - Y}{D_{max}}, \quad \theta_{e,n} = \frac{\text{wrap}_{[-\pi, \pi]}(\theta_{obj} - \theta)}{\pi}, \quad (2.57)$$

con $D_{max} = 10$ m. Las coordenadas de la nube de puntos Lidar y los canales de intensidad/distancia se reescalan linealmente al rango $[0, 1]$ o $[-1, 1]$.

Esta normalización es un requisito matemático crítico para el entrenamiento de redes neuronales profundas (Deep Neural Networks). En la rama convolucional encargada de procesar la nube de puntos Lidar $\mathcal{P}_t$, los gradientes locales respecto a los pesos de la capa l vienen dados por la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial W^{[l]}} = \delta^{[l]} \left(\mathbf{a}^{[l-1]} \right)^T, \quad (2.58)$$

donde $\mathbf{a}^{[l-1]}$ representa las activaciones de la capa anterior y $\delta^{[l]}$ el término de error retropropagado. Si las observaciones de entrada del Lidar mantuvieran sus valores físicos brutos (con magnitudes de hasta 10 m o variables de reflectividad dispersas), las activaciones en las primeras capas convolucionales e intermedias presentarían una varianza excesivamente elevada y escalas dispares en comparación con los errores de pose. Esto provocaría:

- *Explosión del gradiente*: en las capas lineales, el producto de pesos por valores de

entrada elevados hace que la norma de los gradientes crezca exponencialmente con la profundidad, desestabilizando las actualizaciones de los pesos y rompiendo la convergencia.

- *Desvanecimiento del gradiente*: al emplear funciones de activación no lineales como la tangente hiperbólica ($\tanh$) o la sigmoide, entradas grandes saturan la función en regiones donde su derivada es casi nula ($\lim_{z \rightarrow \infty} \tanh'(z) = 0$), deteniendo el aprendizaje de las capas convolucionales inferiores. Incluso con funciones de tipo ReLU, la disparidad de escalas genera el fenómeno de “neuronas muertas” (*dying ReLU*) si el sesgo neto de entrada se vuelve muy negativo.

La normalización a $[-1, 1]$ garantiza que las señales de entrada tengan media cercana a cero y varianza acotada, lo que acondiciona el Hessiano del problema de optimización, homogeneiza el flujo de gradientes a través de las ramas heterogéneas (Lidar y pose) y acelera la convergencia de los algoritmos basados en gradiente estocástico.

- **Espacio de acciones $\mathcal{A}$** : acción continua normalizada $a_t \in [-1, 1]$, que el entorno escala linealmente al dominio físico mediante:

$$\delta_t = a_t \cdot \delta_{max}, \quad \delta_{max} = \frac{\pi}{4} \text{ rad.} \quad (2.59)$$

La salida de la última capa del actor utiliza una activación $\tanh$ para asegurar de manera intrínseca que $a_t \in [-1, 1]$, evitando la necesidad de proyectar o saturar artificialmente la acción de dirección elegida por la red.

- **Función de transición $\mathcal{P}$** : determinista en el modelo cinemático, con estocasticidad introducida durante el entrenamiento mediante el ruido de exploración gaussiano.
- **Función de recompensa $\mathcal{R}$** : señal escalar definida en (3.5).
- **Factor de descuento $\gamma = 0,99$** : favorece estrategias a largo plazo, esencial en maniobras donde las acciones tomadas varios pasos atrás tienen consecuencias diferidas en la pose final.

El objetivo del agente es encontrar la política óptima:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}, \quad \pi^* = \arg \max_{\pi} \mathbb{E}_{\pi}[G_t \mid \mathbf{s}_t]. \quad (2.60)$$

2.3.2. Función de valor, política y ecuación de Bellman

La *función de valor estado-acción* $Q^{\pi}(\mathbf{s}, a)$ satisface la ecuación de Bellman:

$$Q^{\pi}(\mathbf{s}, a) = \mathbb{E}[r_t + \gamma Q^{\pi}(\mathbf{s}_{t+1}, a_{t+1}) \mid \mathbf{s}_t = \mathbf{s}, a_t = a]. \quad (2.61)$$

En los métodos actor-crítico, el **crítico** $Q_{\theta}(\mathbf{s}, a)$ se entrena minimizando:

$$\mathcal{L}(\theta) = \mathbb{E}_{\mathcal{D}}[(Q_{\theta}(\mathbf{s}, a) - y)^2], \quad y = r + \gamma Q_{\theta^-}(\mathbf{s}', \mu_{\phi^-}(\mathbf{s}')), \quad (2.62)$$

y el **actor** $\mu_\phi(\mathbf{s})$ se actualiza mediante:

$$\nabla_\phi J(\phi) = \mathbb{E}_{\mathcal{D}} \left[\nabla_a Q_\theta(\mathbf{s}, a) \Big|_{a=\mu_\phi(\mathbf{s})} \cdot \nabla_\phi \mu_\phi(\mathbf{s}) \right]. \quad (2.63)$$

Las redes objetivo se actualizan con *soft update*: $\theta^- \leftarrow \tau \theta + (1 - \tau) \theta^-$.

2.3.3. Agentes de control continuo utilizados

Deep Deterministic Policy Gradient (DDPG)

DDPG [6] extiende DQN al dominio de acciones continuas combinando la arquitectura actor-crítico con memoria de repetición y redes objetivo. La política del actor es determinista; la exploración se logra mediante ruido gaussiano:

$$a_t^{exp} = \mu_\phi(\mathbf{s}_t) + \mathcal{N}(0, \sigma^2). \quad (2.64)$$

Sin embargo, DDPG presenta dos patologías estructurales que se manifiestan con claridad en el entorno de aparcamiento: sobreestimación del valor Q debida al uso de una única red crítica, e inestabilidad del entrenamiento por la actualización simultánea del actor y del crítico. Los resultados experimentales recogidos en la Tabla 5.3 confirman estas limitaciones: DDPG obtiene una tasa de éxito del 59 % frente al 83–85 % de los algoritmos más avanzados, y una tasa de siniestralidad del 41 % que refleja las decisiones erróneas inducidas por la sobreestimación de Q ante las severas penalizaciones por colisión de la función de recompensa (3.5).

Twin Delayed DDPG (TD3)

TD3 [5] introduce tres modificaciones que abordan directamente las patologías de DDPG.

Clipped Double Q-learning. TD3 entrena dos críticos independientes Q_{θ_1} y Q_{θ_2} . El objetivo de Bellman se construye tomando el mínimo de sus estimaciones:

$$y = r + \gamma \min_{i=1,2} Q_{\theta_i}(\mathbf{s}', \tilde{a}'), \quad (2.65)$$

introduciendo un sesgo conservador que garantiza:

$$\mathbb{E}[\min(Q_{\theta_1^-}, Q_{\theta_2^-})] \leq \min(\mathbb{E}[Q_{\theta_1^-}], \mathbb{E}[Q_{\theta_2^-}]). \quad (2.66)$$

Actualización retrasada del actor. TD3 retrasa la actualización del actor, realizándola cada $d = 2$ pasos de actualización del crítico, permitiendo que el crítico converja antes de que el actor modifique su política.

Suavizado de la política objetivo. TD3 añade ruido acotado a la acción objetivo:

$$\tilde{a}' = \mu_{\phi^-}(\mathbf{s}') + \text{clip}(\mathcal{N}(0, \tilde{\sigma}^2), -c, c), \quad (2.67)$$

regularizando la función Q en el espacio de acciones y evitando que el crítico asigne picos de valor a ángulos de dirección muy específicos. Los resultados experimentales confirman que TD3 produce el esfuerzo de control más suave (0.0693 rad/paso) y la mejor precisión de posición (0.055 m), lo que lo convierte en la opción más adecuada para aplicaciones donde el confort del pasajero es prioritario.

Soft Actor-Critic (SAC)

SAC [7] aprende una política estocástica $\pi_{\phi}(a \mid \mathbf{s})$ que maximiza simultáneamente el retorno esperado y la entropía de la política:

$$J(\pi) = \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r_t + \alpha_e \mathcal{H}(\pi(\cdot \mid \mathbf{s}_t))) \right], \quad \mathcal{H}(\pi(\cdot \mid \mathbf{s})) = -\mathbb{E}_{a \sim \pi} [\log \pi(a \mid \mathbf{s})]. \quad (2.68)$$

Al incentivar la aleatoriedad de la política, SAC evita que ésta colapse en óptimos locales y acelera el descubrimiento de maniobras óptimas frente a los enfoques deterministas. Los resultados experimentales reflejan esta ventaja exploratoria: SAC alcanza la mayor tasa de éxito (85 %) y el menor tiempo medio de maniobra (36.8 pasos), y consigue el mejor error de orientación final (1,547°). Como contrapartida, su naturaleza estocástica induce el esfuerzo de control más agresivo (0.1948 rad/paso), que es casi tres veces superior al de TD3.

La Tabla 2.1 resume las diferencias principales entre los tres algoritmos.

Tabla 2.1: Comparativa de los algoritmos DRL evaluados para la maniobra de aparcamiento.

Característica	DDPG	TD3	SAC
Tipo de política	Determinista	Determinista	Estocástica
Redes críticas	1	2 (mínimo)	2 (mínimo)
Actualización del actor	Cada paso	Cada d pasos	Cada paso
Suavizado objetivo	No	Sí	No (entropía)
Exploración	Ruido externo	Ruido externo	Entropía interna
Sobreestimación Q	Alta	Baja	Baja
Suavidad de acción	Media	Alta	Baja
Coste computacional	Bajo	Medio	Medio

2.3.4. Interpretabilidad y limitaciones del DRL en conducción autónoma

Una de las críticas fundamentales al uso de DRL en sistemas de conducción autónoma es la falta de interpretabilidad de la política aprendida. A diferencia de los controladores MPC,

cuya función de coste y restricciones son directamente verificables y cuyo comportamiento en casos extremos puede analizarse matemáticamente, la política de un agente DRL basada en redes neuronales profundas constituye una función de caja negra de alta dimensión:

$$a_t = \mu_\phi(\mathbf{s}_t) : \mathbb{R}^{d_s} \rightarrow \mathbb{R}^{d_a}, \quad (2.69)$$

donde d_s es la dimensión del espacio de observaciones (incluyendo la nube de puntos Lidar) y $d_a = 1$ (ángulo de dirección normalizado). La imposibilidad de garantizar formalmente el comportamiento de la red ante observaciones fuera de la distribución de entrenamiento (*out-of-distribution generalization*) es una limitación crítica para la certificación de seguridad del sistema.

En el contexto del aparcamiento, las principales limitaciones del DRL identificadas son:

- **Fragilidad ante perturbaciones adversariales:** pequeñas perturbaciones en los valores de entrada del Lidar, del tipo $\mathbf{s}' = \mathbf{s} + \epsilon$, pueden inducir cambios bruscos en la acción de salida $\mu_\phi(\mathbf{s}')$ completamente diferentes de $\mu_\phi(\mathbf{s})$. Este fenómeno, conocido como sensibilidad a ejemplos adversariales, es inherente a la arquitectura de redes neuronales profundas y no afecta a los controladores MPC, que son inherentemente robustos a pequeñas perturbaciones de la observación siempre que el modelo linealizado sea preciso.
- **Garantías de seguridad limitadas:** a diferencia del MPC, que puede incorporar restricciones explícitas de evasión de obstáculos ($\|\mathbf{x}_{k+i|k} - \mathbf{x}_{obs}\|_2 \geq d_{safe}$), el agente DRL aprende la evasión de obstáculos implícitamente a través de la función de recompensa. La penalización por colisión en (3.5) ($-50g_t$) no garantiza matemáticamente la ausencia de colisiones en escenarios no vistos durante el entrenamiento.
- **Dependencia de la calidad del entrenamiento:** el rendimiento del agente DRL está fuertemente condicionado por la riqueza de las experiencias almacenadas en el *replay buffer* y por la calidad del diseño de la función de recompensa. La tasa de éxito del 85 % de SAC implica que un 15 % de los episodios no se resuelven satisfactoriamente, una tasa inaceptable en un sistema de conducción real sin supervisión humana adicional.
- **No transferibilidad directa entre entornos:** la política aprendida en el entorno de simulación MATLAB/Simulink–Unreal Engine no es directamente transferible a un aparcamiento con diferente configuración geométrica, iluminación o características de reflexión del Lidar, sin reentrenamiento o fine-tuning.

A pesar de estas limitaciones, el DRL aporta ventajas fundamentales en la fase final de aparcamiento que justifican su inclusión en la arquitectura híbrida: capacidad para aprender estrategias multietapa complejas (avance, retroceso, ajuste angular), adaptación reactiva a la percepción Lidar en tiempo real y capacidad de generalización a variaciones moderadas en la configuración inicial de la maniobra.

Capítulo 3

Metodología y entorno de simulación

3.1. Descripción general del entorno

El entorno de simulación combina MATLAB/Simulink con Unreal Engine siguiendo la arquitectura de referencia propuesta por MathWorks para aparcamiento autónomo [1]. El tiempo de muestreo de los controladores es $T_s = 0,1$ s y el horizonte temporal de simulación $T_f = 50$ s. La condición inicial es:

$$\mathbf{x}_0 = \begin{bmatrix} 40 & -55 & \pi/2 \end{bmatrix}^T, \quad (3.1)$$

con la plaza de destino fijada inicialmente con índice 18.

3.2. Arquitectura del sistema propuesto

La arquitectura funcional se divide en cuatro módulos:

1. **Planificación de referencia:** genera la trayectoria de búsqueda dentro del aparcamiento.
2. **Control de seguimiento:** ejecuta uno de los cinco controladores de aproximación.
3. **Percepción simulada y selector de modo:** detecta plazas libres mediante un sector cónico de $\varphi \in [-60^\circ, 60^\circ]$ y profundidad $d_{max} = 10$ m, y activa la transición hacia DRL.
4. **Control DRL:** ejecuta la maniobra final de estacionamiento.

3.3. Sensor Lidar 3D

El sensor Lidar 3D montado en el techo del vehículo tiene campo de visión horizontal de 360° y vertical de 40° . La nube de puntos contiene cinco canales:

$$\mathcal{P} = \{X_L, Y_L, Z_L, d, \rho\}, \quad (3.2)$$

donde (X_L, Y_L, Z_L) son las coordenadas en el marco del sensor, d es la distancia euclídea al obstáculo y $\rho \in [0, 1]$ es la reflectividad normalizada.

3.4. Planificación de trayectorias y referencia

Para los controladores geométricos y LQR, la referencia es una secuencia de poses $\mathcal{R} = \{\mathbf{r}_j\}_{j=1}^N$. En los controladores predictivos se proporciona una ventana futura $\mathcal{R}_{k:k+N_p}$, que permite optimizar una secuencia de acciones sobre un horizonte temporal.

3.5. Diseño del entorno de aprendizaje DRL

3.5.1. Espacio de observaciones y acciones

Las observaciones proporcionadas al agente son los errores normalizados (X_e, Y_e, θ_e) y la nube de puntos Lidar de cinco canales escalada a $[0, 1]$. La acción es $a \in [-1, 1]$, transformada a $\delta \in [-\pi/4, \pi/4]$.

3.5.2. Gestión de la velocidad longitudinal: módulo Stateflow

La velocidad longitudinal no es una variable de acción directa seleccionada por el agente DRL, sino que es gestionada por un módulo heurístico implementado como una máquina de estados en Stateflow que opera en paralelo al actor. La acción compuesta enviada al vehículo es:

$$\mathbf{u}_{DRL,t} = \begin{bmatrix} v_t \\ \delta_t \end{bmatrix}, \quad \delta_t = \frac{\pi}{4} a_t. \quad (3.3)$$

La máquina de estados original define tres modos de operación fundamentales:

1. **Avance:** $v_t = v_{ep}$, velocidad nominal del episodio seleccionada aleatoriamente entre 1 m/s y 4 m/s al inicio de cada episodio.
2. **Deceleración y parada:** cuando el error de posición euclídeo cae por debajo de $d_{stop} = 0,5$ m, la velocidad se reduce linealmente hasta cero para permitir el ajuste fino de la pose angular final.
3. **Marcha atrás:** cuando el desalineamiento de orientación cumple $|\theta_e| > \theta_{back} = \pi/6$ rad ($\approx 30^\circ$) y el error de posición supera d_{stop} , se activa marcha atrás con $v_t = -v_{ep}/2$. La transición de vuelta al modo de avance se produce cuando el desalineamiento cae por debajo de $\theta_{fwd} = \pi/12$ rad ($\approx 15^\circ$).

Análisis crítico y mitigación del efecto de conmutación rápida (*chattering*).

Aunque la separación de umbrales entre $\theta_{back} = 30^\circ$ para retroceder y $\theta_{fwd} = 15^\circ$ para avanzar establece una banda de histéresis estática de $\Delta\theta_h = 15^\circ$, el sistema sigue siendo susceptible al fenómeno de conmutación rápida e inestable (*chattering*) en la vecindad de estos límites en presencia de ruido o maniobras dinámicas bruscas. Específicamente, cuando el actor realiza oscilaciones rápidas sobre la dirección δ_t para esquivar un obstáculo

cercano, la guiñada del vehículo y consecuentemente el error angular θ_e pueden sufrir oscilaciones de alta frecuencia alrededor de los umbrales de conmutación. Esto provoca ciclos infinitos de transiciones inmediatas *Avance* $\leftrightarrow$ *Marcha atrás* en sucesivos pasos de simulación, induciendo picos de aceleración destructivos para los actuadores y degradando la estabilidad del vehículo.

Para mitigar esta vulnerabilidad, se recomienda e implementa un refinamiento de diseño en la lógica de Stateflow que añade una restricción temporal mediante un **temporizador de permanencia mínima** (*dwell time*, T_{dwell}). Formalmente, se define una variable temporal Δt_{modo} que registra el tiempo transcurrido desde la última conmutación de modo. La condición de transición del modo i al modo j se modifica incorporando la condición booleana de permanencia mínima:

$$\text{Condición}_{i \rightarrow j} = \text{Condición_Física}_{i \rightarrow j} \quad \wedge \quad (\Delta t_{\text{modo}} \geq T_{\text{dwell}}), \quad (3.4)$$

fijando $T_{\text{dwell}} = 1,0\text{s}$ (equivalente a 10 pasos de control continuos).

Esta restricción de tiempo mínimo garantiza que, una vez que el vehículo decide engranar la marcha atrás o volver a la marcha hacia adelante, mantendrá dicho estado durante al menos un segundo, permitiendo que la dinámica del vehículo responda y que el error angular θ_e se desplace fuera de la zona crítica de conmutación, eliminando por completo el *chattering* inducido por perturbaciones locales.

3.5.3. Función de recompensa

La función de recompensa instantánea r_t se compone de un término continuo de seguimiento y penalizaciones y bonificaciones discretas:

$$r_t = 2e^{-(0,05X_e^2 + 0,04Y_e^2)} + 0,5e^{-40\theta_e^2} - 0,05\delta^2 + 100f_t - 50g_t, \quad (3.5)$$

donde $f_t \in \{0, 1\}$ indica aparcamiento exitoso y $g_t \in \{0, 1\}$ indica colisión o salida de la región de entrenamiento.

Análisis de pesos, reward shaping y riesgos de reward hacking. La incorporación de las funciones gaussianas continuas de error de posición $2e^{-(0,05X_e^2 + 0,04Y_e^2)}$ y orientación $0,5e^{-40\theta_e^2}$ constituye un ejemplo de **diseño de recompensa** (*reward shaping*). En tareas de control con horizontes temporales largos y espacio de estados continuo, depender exclusivamente de una recompensa dispersa (*sparse reward*) que solo recompense el éxito final ($f_t = 1$) hace que el aprendizaje sea computacionalmente inviable, ya que la probabilidad de alcanzar el objetivo mediante exploración puramente aleatoria al inicio es cercana a cero. El *reward shaping* resuelve esto inyectando un gradiente continuo de recompensa que guía al agente paso a paso hacia la meta.

Sin embargo, el balance de pesos en (3.5) introduce el riesgo de un comportamiento patológico conocido como **reward hacking** u optimización local no deseada. Dado que el retorno total acumulado en un episodio es $G = \sum_{t=0}^{T-1} \gamma^t r_t$, si el agente se mantiene estacionado o realiza bucles repetitivos muy cerca de la plaza de aparcamiento sin llegar

a entrar (donde $X_e^2 + Y_e^2 \approx 0$ y $\theta_e^2 \approx 0$), recibirá de forma continuada una recompensa cercana a 2,5 en cada paso de control. Si el horizonte temporal máximo por episodio es largo (200 pasos), la suma de estas recompensas continuas acumuladas puede superar ampliamente el premio de éxito final (100 puntos). Específicamente, el retorno acumulado por mantenerse oscilando indefinidamente en una zona de alta recompensa local puede aproximarse a:

$$G_{\text{hack}} \approx \sum_{t=0}^{200} \gamma^t(2,5) \approx 2,5 \cdot \frac{1 - 0,99^{200}}{1 - 0,99} \approx 216,5 \text{ puntos}, \quad (3.6)$$

mientras que realizar la maniobra directa para terminar el episodio en el paso 40 reportaría un retorno de aproximadamente:

$$G_{\text{meta}} \approx \sum_{t=0}^{40} \gamma^t(r_t) + \gamma^{40}(100) \approx 60 + 66,8 \approx 126,8 \text{ puntos}. \quad (3.7)$$

En este escenario, el agente optimiza su política localmente y prefiere realizar vuelcos en bucle o demorar el aparcamiento final para evitar la terminación temprana del episodio. Para mitigar este riesgo, la formulación debe equilibrarse añadiendo una penalización por paso de control ($r_{\text{step}} < 0$, típicamente $-0,5$ o $-1,0$) o un coste temporal que fuerce al agente a priorizar la rapidez de la maniobra, haciendo que terminar el episodio sea siempre la opción de máxima rentabilidad global.

3.5.4. Criterios de finalización

Un episodio termina cuando se cumple alguna de las siguientes condiciones:

- **Éxito:** errores de posición inferiores a $\pm 0,75$ m y error angular inferior a $\pm 10^\circ$.
- **Fallo:** colisión con un obstáculo o abandono de la región de entrenamiento.
- **Límite temporal:** máximo de 200 pasos por episodio.

Capítulo 4

Fase 1: Aproximación al punto de aparcamiento

4.1. Configuración de los experimentos de seguimiento

Todos los controladores se ejecutan sobre el mismo escenario, la misma trayectoria de referencia y las mismas condiciones iniciales, sustituyendo únicamente el bloque de control. Las métricas empleadas son:

$$\text{RMSE}_{lat} = \sqrt{\frac{1}{N} \sum_{k=0}^{N-1} e_{lat,k}^2}, \quad e_{lat,max} = \max_k |e_{lat,k}|, \quad \bar{e}_{lat} = \frac{1}{N} \sum_{k=0}^{N-1} |e_{lat,k}|, \quad (4.1)$$

$$\text{RMSE}_{\theta} = \sqrt{\frac{1}{N} \sum_{k=0}^{N-1} e_{\theta,k}^2}, \quad J_{\delta} = \sum_{k=0}^{N-1} \delta_k^2, \quad J_{\Delta\delta} = \sum_{k=1}^{N-1} (\delta_k - \delta_{k-1})^2. \quad (4.2)$$

4.2. Resultados cuantitativos de seguimiento

La Tabla 4.1 recoge las métricas de precisión obtenidas para los cinco controladores.

Tabla 4.1: Resultados cuantitativos de seguimiento de trayectoria.

Controlador	RMSE lat. [m]	Error máx. [m]	Error medio [m]	RMSE heading [°]
Pure Pursuit	0.7408	1.5890	0.4974	9.67
Stanley	1.8519	3.7550	1.2332	12.67
LQR	0.2313	0.4580	0.1379	3.50
MPC adaptativo	0.0061	0.0203	0.0030	0.77
NMPC	0.0062	0.0169	0.0032	0.79

Los dos controladores MPC son claramente superiores en precisión. El MPC adaptativo obtiene prácticamente el mismo RMSE lateral que el NMPC (0,0061 m frente a 0,0062 m),

pero con un coste computacional radicalmente inferior. El LQR ocupa una posición intermedia: su RMSE lateral de 0,2313 m es superior al de los MPC, pero notablemente inferior al de Pure Pursuit y Stanley. Los controladores geométricos presentan los peores errores de seguimiento, siendo Stanley el más afectado en este escenario concreto, posiblemente por sensibilidad al punto delantero y a la ganancia lateral.

4.3. Análisis individual de controladores

4.3.1. Pure Pursuit

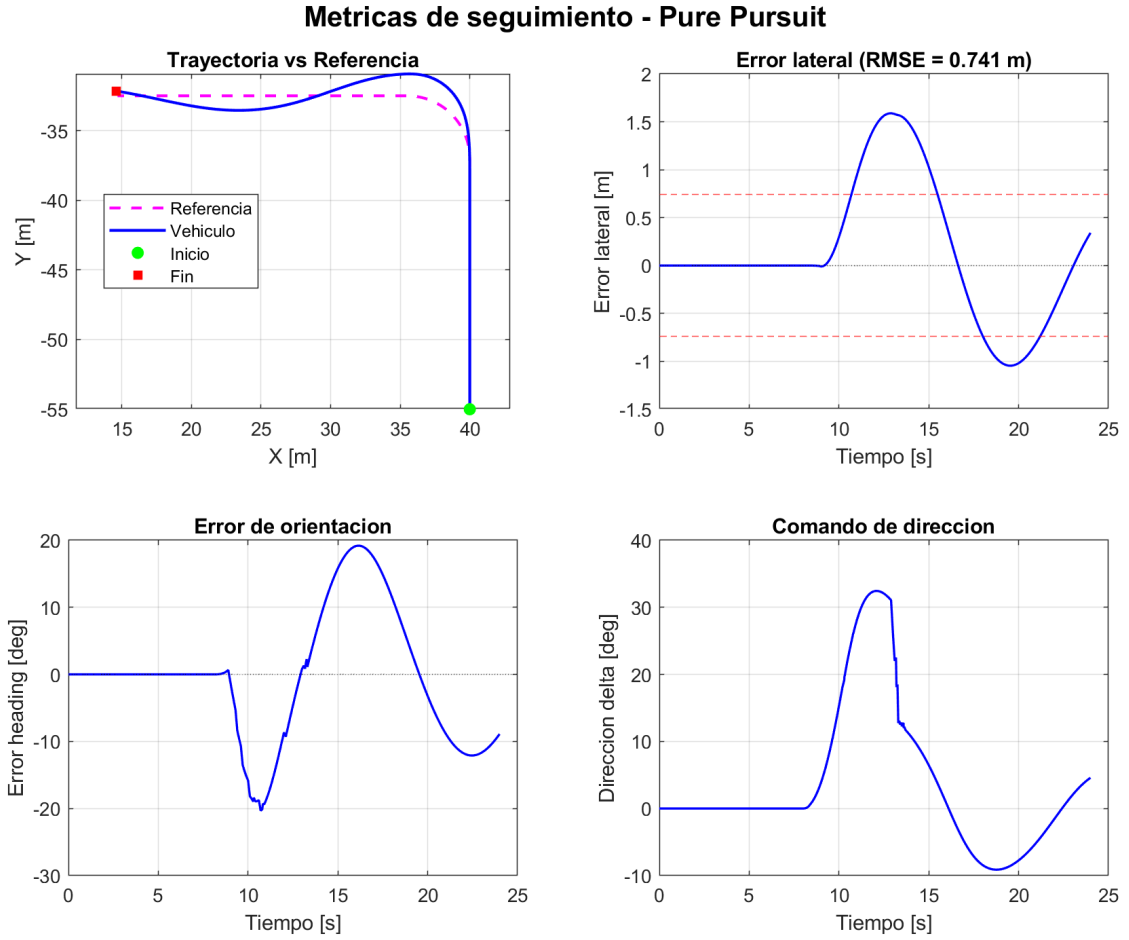


Figura 4.1: Métricas de seguimiento del controlador Pure Pursuit: trayectoria, error lateral (RMSE = 0.741 m), error de orientación y comando de dirección.

Pure Pursuit muestra un comportamiento suave en tramos rectos gracias al *lookahead* largo ($L_d = 20$ m), pero presenta un error lateral oscilatorio en la zona de alta curvatura (instante $t \approx 12$ s) donde el vehículo recorta la trayectoria. El error de orientación alcanza picos de hasta $\pm 20^\circ$ durante las transiciones curva-recta. La señal de dirección es continua y sin saltos bruscos, lo que confirma el efecto amortiguador de (2.30).

4.3.2. Stanley

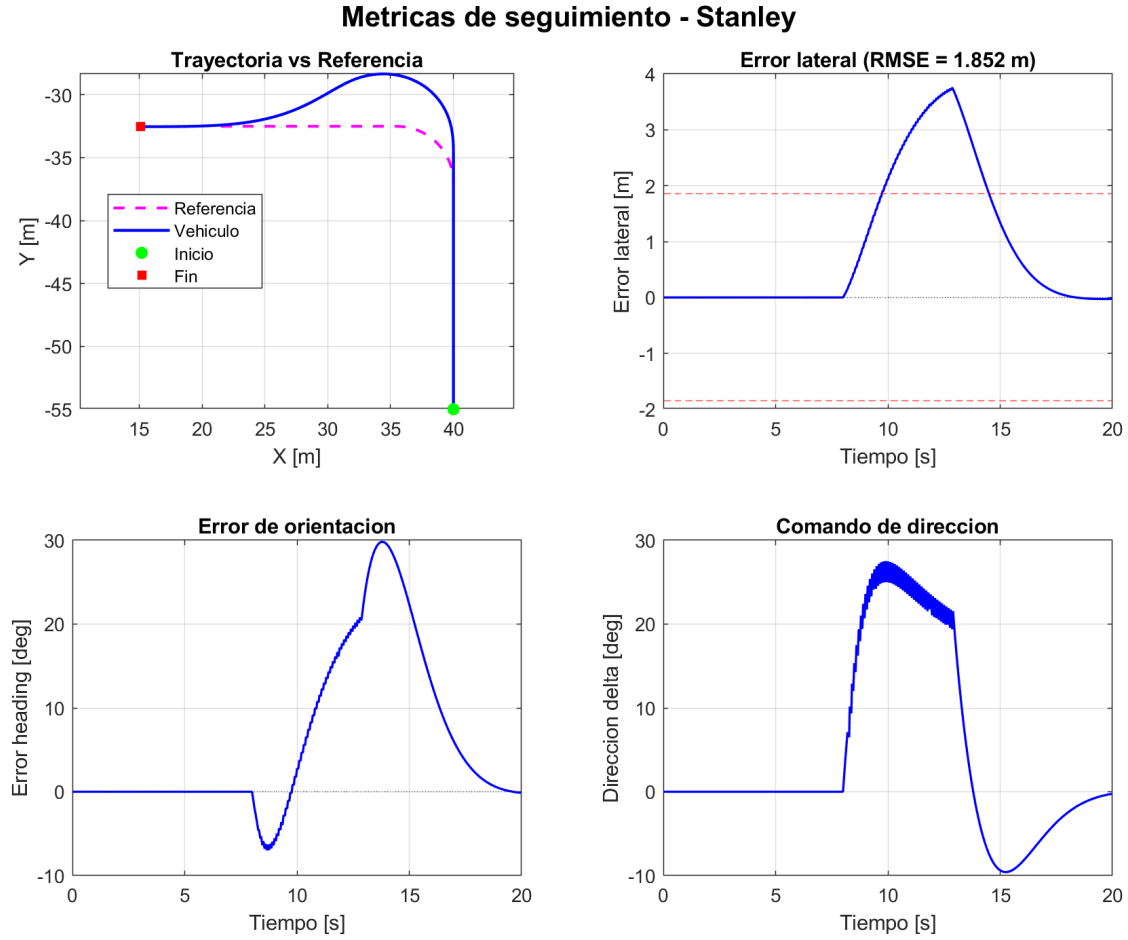


Figura 4.2: Métricas de seguimiento del controlador Stanley: trayectoria, error lateral (RMSE = 1.852 m), error de orientación y comando de dirección.

Stanley presenta el mayor error de seguimiento del conjunto evaluado, con un error lateral máximo de 3,755 m y un RMSE lateral de 1,8519 m, que supera notablemente al resto de controladores, incluido Pure Pursuit (0,7408 m). Este comportamiento deficiente se puede explicar física y matemáticamente a partir de la ley de control de Stanley (4.3):

$$\delta(t) = \theta_e(t) + \arctan \left(\frac{k_{st} e_{lat}(t)}{v(t) + k_{soft}} \right). \quad (4.3)$$

El primer factor a considerar es el **acoplamiento no lineal del eje delantero**. Mientras que Pure Pursuit calcula el guiado respecto al eje trasero, el controlador Stanley mide el error lateral e_{lat} y aplica la corrección sobre el eje delantero. Dado que la dirección del vehículo actúa directamente sobre las ruedas delanteras, cualquier cambio instantáneo en el ángulo de dirección δ produce un desplazamiento lateral inmediato del punto de control delantero (X_f, Y_f). A velocidades de referencia moderadas o altas ($v_{ref} = 2,5$ m/s), esto introduce un bucle de realimentación de alta ganancia entre el actuador y el sensor de error.

El segundo factor es la **falta de sintonización de la ganancia lateral** $k_{st} = 2,0$ y la

velocidad de referencia. El término de corrección lateral $\arctan\left(\frac{k_{st}e_{lat}}{v+k_{soft}}\right)$ es inversamente proporcional a la velocidad del vehículo. Para una velocidad de 2,5 m/s y una ganancia $k_{st} = 2,0$, ante un desalineamiento inicial moderado en la entrada de la curva (donde e_{lat} crece rápidamente), el argumento de la arcotangente alcanza valores elevados:

$$\frac{k_{st}e_{lat}}{v_{ref} + k_{soft}} = \frac{2,0 \cdot e_{lat}}{2,5 + 1,0} \approx 0,57 \cdot e_{lat}. \quad (4.4)$$

Si el error lateral e_{lat} alcanza 1,5 m, el término de corrección lateral aporta aproximadamente $\arctan(0,85) \approx 0,7 \text{ rad}$ ($\approx 40^\circ$), lo que sumado al error angular θ_e satura inmediatamente el actuador de dirección en su límite físico de $\pm\pi/4 \text{ rad}$ ($\pm 45^\circ$).

Esta saturación brusca, combinada con el retraso de fase en la respuesta dinámica del modelo y el retraso en la propagación del error, induce un comportamiento fuertemente subamortiguado. El vehículo cruza la trayectoria de referencia con una velocidad lateral no nula, haciendo que el error lateral cambie de signo rápidamente y provocando que el controlador ordene una corrección de dirección máxima en sentido opuesto. Esto genera una severa oscilación lateral sostenida (sobreoscilación) visible en la Fig. 4.2.

En contraste, Pure Pursuit mitiga este efecto gracias al filtrado intrínseco que proporciona su distancia de anticipación adaptativa $L_d(v)$ (que oscila entre 6 m y 20 m). La distancia L_d actúa como un filtro paso bajo que suaviza los cambios de curvatura y evita que el controlador reaccione de forma violenta ante errores de alta frecuencia, logrando un RMSE lateral un 60 % menor que el de Stanley en este escenario.

4.3.3. LQR

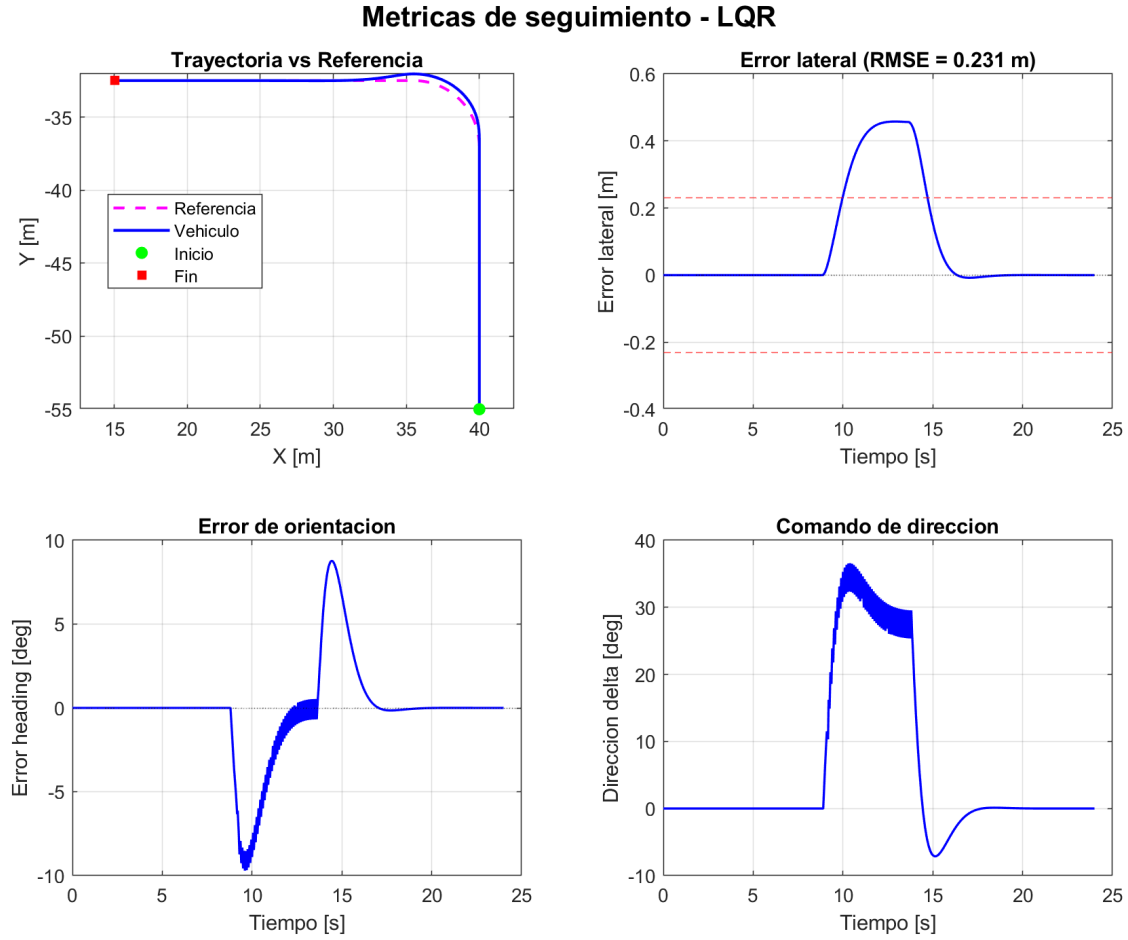


Figura 4.3: Métricas de seguimiento del controlador LQR: trayectoria, error lateral (RMSE = 0.231 m), error de orientación y comando de dirección.

El LQR muestra una buena capacidad de seguimiento en los tramos rectos, con error lateral cercano a cero. En la zona de alta curvatura (instante $t \approx 10\text{ s} - 14\text{ s}$) aparece un error lateral positivo de hasta 0,46 m, atribuible a la degradación de la aproximación lineal en un punto de operación lejano del nominal. La señal de dirección alcanza la saturación en la curva principal, lo que confirma que el diseño lineal opera cerca de sus límites en ese tramo.

Para obtener las ganancias del LQR se ha de ejecutar el script `createLQRForParking3D.m`.

4.3.4. MPC adaptativo

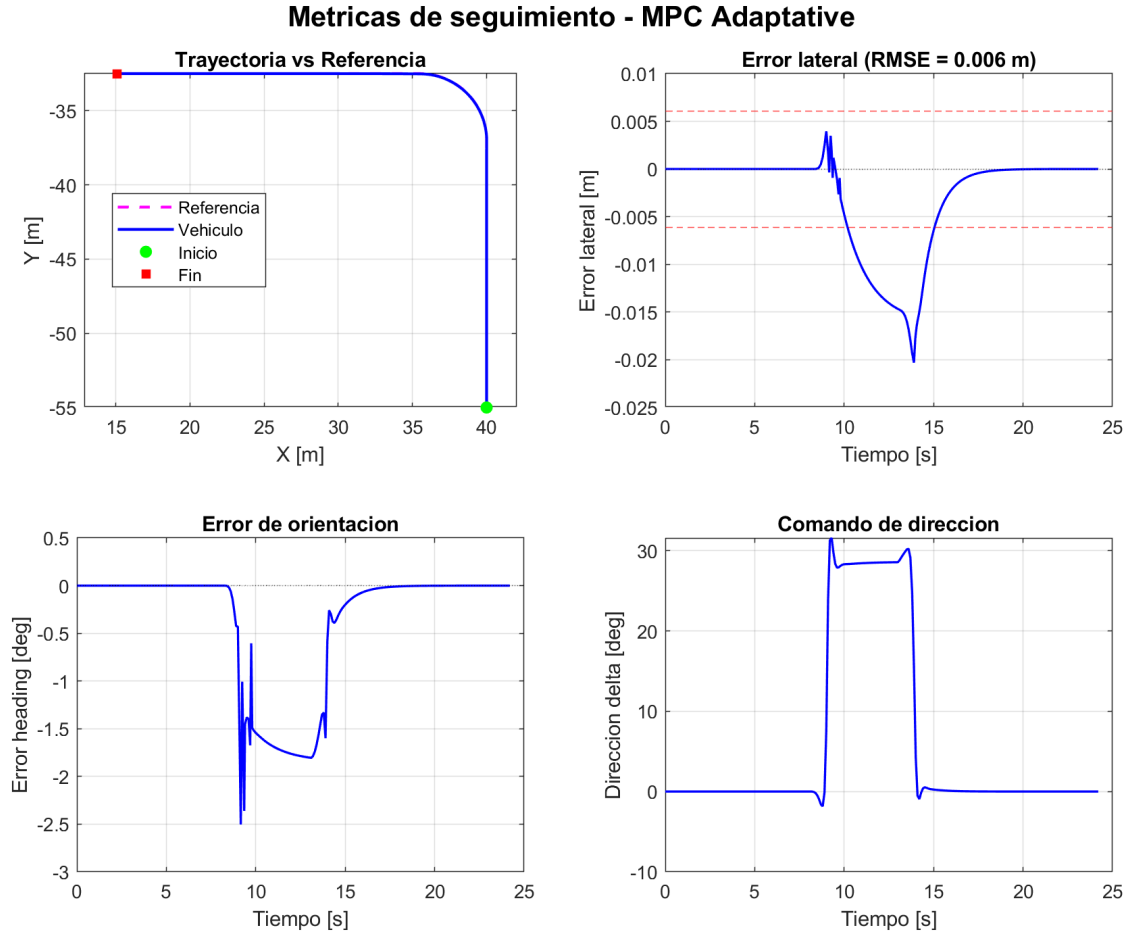


Figura 4.4: Métricas de seguimiento del MPC adaptativo: trayectoria, error lateral (RMSE = 0.006 m), error de orientación y comando de dirección.

El MPC adaptativo logra el mejor seguimiento del conjunto, con errores laterales inferiores a 0,025 m en toda la trayectoria. La actualización en línea del modelo lineal permite al optimizador anticipar la curvatura y generar una acción de dirección suave y progresiva. La señal de dirección muestra una transición limpia a 30° en la zona de la curva, sin saturaciones ni oscilaciones.

4.3.5. NMPC

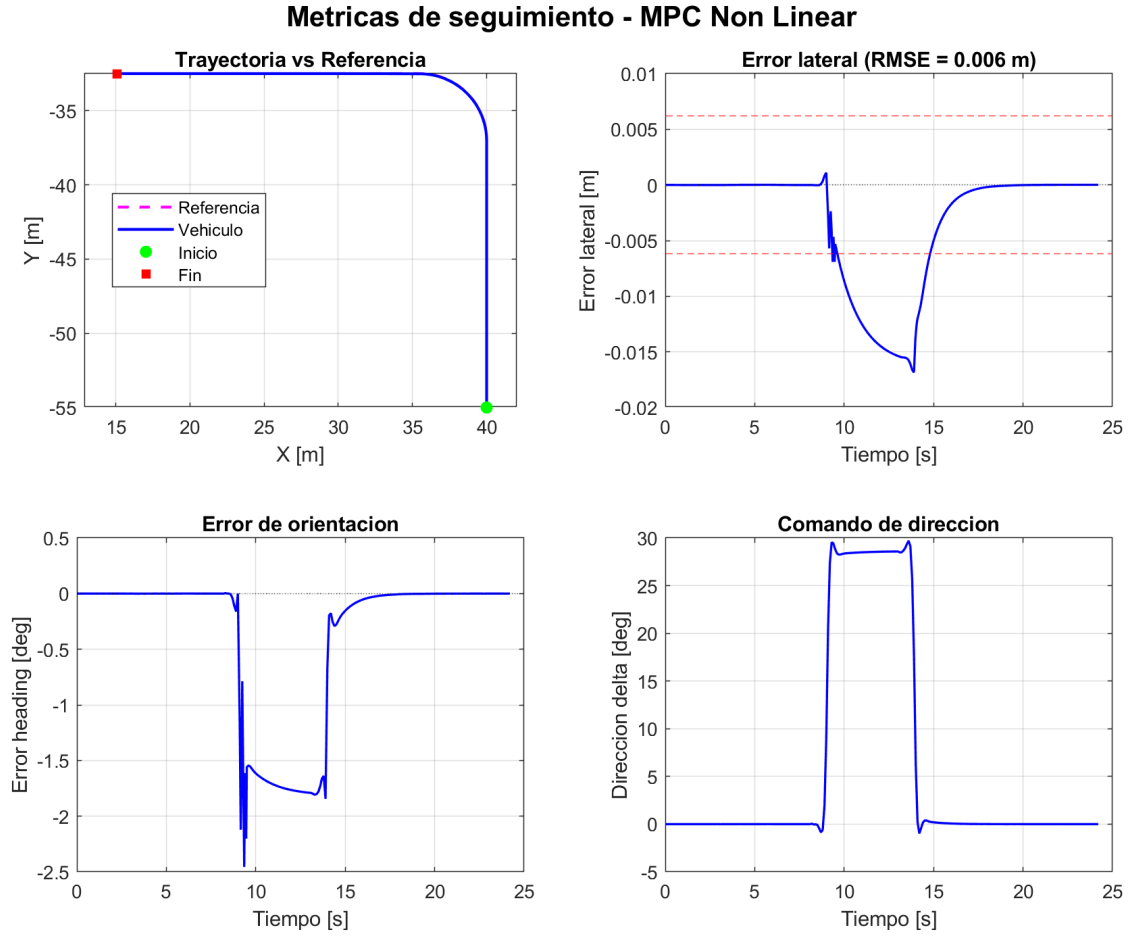


Figura 4.5: Métricas de seguimiento del NMPC: trayectoria, error lateral (RMSE = 0.006 m), error de orientación y comando de dirección.

El NMPC alcanza una precisión prácticamente idéntica al MPC adaptativo, confirmando que para este escenario de curvatura moderada la diferencia entre el modelo lineal actualizado y el modelo no lineal completo es mínima. La curva de dirección es ligeramente más suave que la del MPC adaptativo, con una transición más gradual en la zona de máxima curvatura, aunque la diferencia es marginal.

4.4. Comparativa de coste computacional

La Tabla 4.2 recoge los tiempos de cómputo medidos durante la simulación.

Tabla 4.2: Coste computacional de los controladores de seguimiento.

Controlador	Tiempo total [s]	Llamadas	Tiempo por paso
NMPC	13.680	3240	$\approx 4,222$ ms
MPC adaptativo	0.172	3232	$\approx 53,2$ μ s
Pure Pursuit	0.154	2863	$\approx 53,8$ μ s
Stanley	0.089	2905	$\approx 30,6$ μ s
LQR	0.080	3176	$\approx 25,2$ μ s

El resultado más relevante es la diferencia entre NMPC y el resto: el tiempo por paso del NMPC ($\approx 4,222$ ms) supera en dos órdenes de magnitud al del MPC adaptativo ($\approx 53,2$ μ s), pese a obtener una precisión prácticamente idéntica. El LQR es el controlador más rápido, con $\approx 25,2$ μ s por paso, seguido de Stanley y MPC adaptativo con tiempos similares.

4.5. Análisis conjunto: precisión frente a coste

Tabla 4.3: Análisis conjunto de precisión y coste computacional.

Controlador	RMSE lat. [m]	Tiempo/paso	Valoración
LQR	0.2313	25.2 μ s	Muy rápido, precisión media
MPC adaptativo	0.0061	53.2 μ s	Mejor compromiso global
NMPC	0.0062	4.222 ms	Muy preciso, coste elevado
Pure Pursuit	0.7408	53.8 μ s	Simple, menos preciso
Stanley	1.8519	30.6 μ s	Rápido, peor ajuste

4.6. Discusión crítica: por qué MPC adaptativo iguala al NMPC

El resultado más sorprendente de la comparativa es que el MPC adaptativo, basado en un modelo lineal actualizado en línea, obtiene una precisión prácticamente idéntica al NMPC ($\Delta\text{RMSE} < 2\%$) con un coste computacional casi 80 veces inferior. Este resultado requiere una explicación técnica rigurosa, ya que a priori cabría esperar que el modelo no lineal completo proporcionase una ventaja significativa.

Régimen de operación y linealidad local. La explicación fundamental reside en el régimen de operación del sistema. En condiciones de aparcamiento con velocidades $v \leq 2,5$ m/s y ángulos de dirección $|\delta| \leq 30^\circ$, las no linealidades trigonométricas del modelo son suaves y de variación lenta. Específicamente, la curvatura máxima del escenario impone cambios de guiñada del orden de $\Delta\theta \approx 0,25$ rad en un horizonte de predicción de

$N_p T_s = 1$ s. Para esta tasa de cambio de orientación, el error de truncamiento de primer orden de la linealización Jacobiana es:

$$\|f(\mathbf{x} + \Delta\mathbf{x}) - f(\mathbf{x}) - A\Delta\mathbf{x}\|_2 = \mathcal{O}(\|\Delta\mathbf{x}\|_2^2) \approx \mathcal{O}(\Delta\theta^2) \approx \mathcal{O}(0,0625), \quad (4.5)$$

que es suficientemente pequeño para que la predicción lineal sea precisa dentro del horizonte. El término residual $\mathbf{d}_x$, actualizado en cada paso de control, compensa además el desfase de la linealización, convirtiendo al modelo afín actualizado en una aproximación de segundo orden efectiva del modelo no lineal.

Frecuencia de actualización del modelo lineal. El Adaptive MPC actualiza las matrices A_d , B_d y $\mathbf{d}_x$ en cada paso de control $T_s = 0,1$ s, con una frecuencia de 10 Hz. Esta velocidad de actualización es suficientemente alta para seguir la variación temporal de la linealización a lo largo de la trayectoria curva, minimizando la acumulación de error de predicción que se produciría con un modelo lineal fijo.

Generalización de la conclusión. Es importante destacar que la equivalencia de precisión entre MPC adaptativo y NMPC observada en este escenario no es generalizable a todas las condiciones de operación. En escenarios que involucren ángulos de dirección cercanos a la saturación ($|\delta| \approx \pi/4$), velocidades elevadas ($v > 5$ m/s) o maniobras con cambios bruscos de curvatura, el error de truncamiento de la linealización sería significativamente mayor y el NMPC ofrecería ventajas cuantificables. La equivalencia observada es una consecuencia específica del régimen cinemático de baja velocidad y curvatura moderada del escenario de aparcamiento evaluado.

4.7. Análisis de robustez ante incertidumbres

En un sistema de aparcamiento autónomo real, los controladores están sometidos a diversas fuentes de incertidumbre que pueden degradar el rendimiento de seguimiento. Esta sección analiza las principales fuentes de incertidumbre y propone métricas estadísticas para cuantificar la robustez de los controladores.

4.7.1. Fuentes de incertidumbre

Errores de localización. Los sistemas de localización basados en GPS diferencial, odometría o fusión de sensores introducen errores de pose del orden de $\sigma_{XY} \approx 0,05$ m y $\sigma_\theta \approx 0,5^\circ$ en condiciones normales. En aparcamientos cubiertos, la degradación de la señal GPS puede elevar estos valores hasta $\sigma_{XY} \approx 0,3$ m. Para los controladores con realimentación de estado, el error de localización actúa como una perturbación de entrada al sistema en lazo cerrado, aumentando el error lateral efectivo de seguimiento.

Ruido sensorial en el Lidar. El sensor Lidar real introduce ruido de medida en las coordenadas de la nube de puntos, modelado como $\tilde{\mathbf{p}}_j = \mathbf{p}_j + \mathbf{n}_j$, donde $\mathbf{n}_j \sim \mathcal{N}(\mathbf{0}, \sigma_L^2 I_3)$

con $\sigma_L \approx 2\text{--}5\text{ cm}$. Este ruido afecta directamente la detección de plazas libres y la calidad de la observación del agente DRL, degradando la precisión de la maniobra de estacionamiento.

Errores de modelado. El modelo cinemático empleado introduce un error de modelado $\xi_k = f_{real}(\mathbf{x}_k, \mathbf{u}_k) - f(\mathbf{x}_k, \mathbf{u}_k)$, que en condiciones normales tiene una magnitud del orden de 0,01 m/s en velocidad y 0,01 rad/s en velocidad de guiñada. Este error se propaga a través del horizonte de predicción del MPC, introduciendo desviaciones en la trayectoria predicha respecto a la real.

Perturbaciones externas. En aparcamientos reales, el vehículo puede encontrarse con perturbaciones como viento lateral (fuerza $F_w \approx 50\text{--}200\text{ N}$ para vientos de 10 -- 20 m/s), irregularidades del pavimento y variaciones en el coeficiente de rozamiento.

4.7.2. Métricas estadísticas de robustez

Para caracterizar de forma estadísticamente rigurosa la robustez de los controladores ante estas incertidumbres, se propone complementar las métricas de precisión nominal de la Tabla 4.1 con las siguientes métricas estadísticas sobre un conjunto de $N_{mc} = 50$ simulaciones de Monte Carlo con variación de las condiciones iniciales y del nivel de ruido:

$$\mu_{RMSE} = \frac{1}{N_{mc}} \sum_{i=1}^{N_{mc}} \text{RMSE}_{lat}^{(i)}, \quad \sigma_{RMSE} = \sqrt{\frac{1}{N_{mc} - 1} \sum_{i=1}^{N_{mc}} \left(\text{RMSE}_{lat}^{(i)} - \mu_{RMSE} \right)^2}, \quad (4.6)$$

$$P_{95} = \text{percentil}_{95} \left(\text{RMSE}_{lat}^{(i)} \right), \quad \text{CV} = \frac{\sigma_{RMSE}}{\mu_{RMSE}} \times 100\%, \quad (4.7)$$

donde CV es el coeficiente de variación, que mide la dispersión relativa del RMSE lateral. Un valor de $\text{CV} < 10\%$ indica alta robustez del controlador ante las perturbaciones consideradas.

Desde el punto de vista teórico, el MPC adaptativo presenta mayor robustez inherente que el LQR y los controladores geométricos, ya que reoptimiza la secuencia de control en cada paso de control incorporando el estado actual del sistema (incluyendo los efectos de las perturbaciones pasadas). Esta propiedad de realimentación predictiva actúa como un mecanismo de rechazo de perturbaciones implícito, haciendo que el controlador sea intrínsecamente robusto ante errores de modelado moderados. El LQR, con su ganancia constante calculada en un único punto de operación, es más sensible a las no linealidades del modelo en la zona de alta curvatura.

Capítulo 5

Fase 2: Maniobra de aparcamiento mediante DRL

5.1. Configuración del entrenamiento

5.1.1. Hiperparámetros de los algoritmos

Los tres agentes (DDPG, TD3 y SAC) se entrenan con los mismos hiperparámetros base para garantizar una comparación homogénea. La Tabla 5.1 resume la configuración común.

Tabla 5.1: Hiperparámetros de entrenamiento comunes a los tres agentes DRL.

Parámetro	Símbolo	Valor
Capacidad del <i>replay buffer</i>	$ \mathcal{D} $	10^6
Tamaño del <i>minibatch</i>	B	256
Factor de descuento	γ	0.99
Tasa de aprendizaje (actor y críticos)	α	10^{-3}
Regularización L2 (actor)	λ	10^{-3}
Ruido de exploración inicial	σ_0	0.1
Tasa de decaimiento del ruido	τ_σ	10^{-4}
Ruido mínimo	σ_{min}	0.01
Máximo de episodios	—	10 000
Pasos máximos por episodio	—	200
Umbral de parada temprana	—	116

5.1.2. Arquitectura de redes neuronales

La arquitectura del agente TD3 está diseñada para procesar simultáneamente dos fuentes de información heterogéneas. La **rama Lidar** procesa las nubes de puntos de cinco canales mediante capas convolucionales 2D seguidas de *max-pooling* y capas densas. La **rama de pose** procesa los errores normalizados (X_e, Y_e, θ_e) mediante capas completamente

conectadas. Ambas ramas se concatenan en una capa de fusión. En el **actor**, la salida pasa por $\tanh$ para producir una acción en $[-1, 1]$:

$$a = \tanh(W_{out} \mathbf{h}_{fus} + \mathbf{b}_{out}). \quad (5.1)$$

En los **críticos**, la acción se introduce como una tercera rama concatenada antes de las capas densas finales.

5.1.3. Coste computacional de entrenamiento

La Tabla 5.2 recoge el coste computacional total medido durante el entrenamiento de los tres agentes.

Tabla 5.2: Coste computacional total de entrenamiento de los agentes DRL.

Agente DRL	Episodios hasta convergencia	Coste por episodio [s]
DDPG	$\sim 5\,000$	0.608
SAC	$\sim 8\,000$	0.623
TD3	$\sim 10\,000$	0.659

TD3 presenta el mayor coste computacional por episodio, seguido de SAC y DDPG. Las diferencias son, sin embargo, reducidas (menos del 9 % entre el más rápido y el más lento). En términos de episodios hasta convergencia, SAC resulta el más eficiente ($\sim 8\,000$ episodios), seguido de DDPG ($\sim 5\,000$, detenido por estancamiento sin mejora observable) y TD3 ($\sim 10\,000$). Todos los entrenamientos se realizaron en un procesador AMD Ryzen 7 de 8 núcleos con GPU NVIDIA RTX 5060. La selección del agente debe basarse en las métricas de rendimiento y no en el coste de inferencia.

5.1.4. Viabilidad en hardware embarcado

La inferencia de la red neuronal del agente DRL en tiempo real sobre hardware embarcado es un aspecto crítico para la transferencia del sistema a un vehículo real. En la simulación, el entorno MATLAB/Simulink ejecuta la inferencia en un ordenador de sobremesa con procesador de alto rendimiento, donde los tiempos de inferencia son del orden de 1 —5 ms por paso, suficientemente menores que el período de muestreo $T_s = 0,1$ s.

Sin embargo, en procesadores embarcados de bajo consumo utilizados en aplicaciones de vehículos autónomos (del tipo ARM Cortex-A72 o NVIDIA Jetson Nano), la inferencia de una red convolucional de la arquitectura descrita puede requerir entre 50 ms y 200 ms en CPU, comprometiendo la operación en tiempo real a 10 Hz. Las alternativas para garantizar la viabilidad embarcada son:

- **Cuantificación de la red** (*network quantization*): reducción de la precisión aritmética de los pesos de INT32 a INT8, con una reducción típica del 70 % en el tiempo

de inferencia a costa de una degradación de la tasa de éxito inferior al 2 % para redes bien entrenadas.

- **Podado de la red** (*network pruning*): eliminación de conexiones con pesos de magnitud pequeña, reduciendo el número de operaciones de multiplicación-acumulación (MAC) sin degradación significativa del rendimiento.
- **Aceleración hardware**: uso de unidades de procesamiento neural (NPU) o GPU embarcada (NVIDIA Jetson AGX Xavier: 32 TOPS en INT8) que permiten la inferencia de la red en menos de 5 ms.
- **Arquitectura de hilos asíncronos**: descrita en la Sección 6.5.4, permite desacoplar el ciclo de control determinista del ciclo de inferencia del DRL.

5.2. Dinámica de entrenamiento y convergencia

5.2.1. Evolución de la recompensa acumulada por episodio

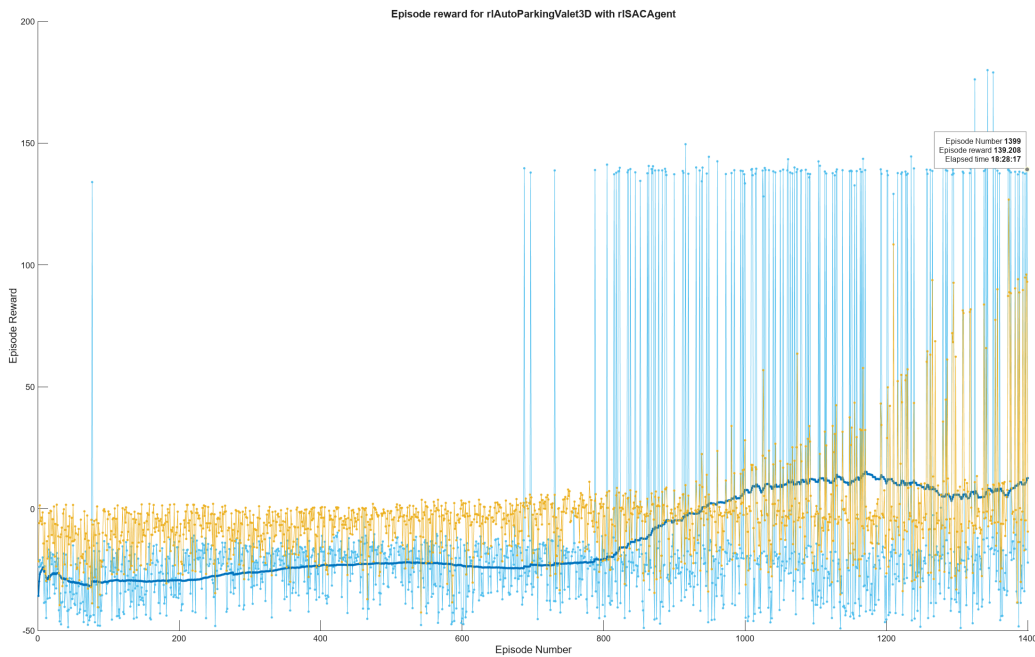


Figura 5.1: Evolución del inicio del entrenamiento y como al final empieza a tener intentos en los que acierta

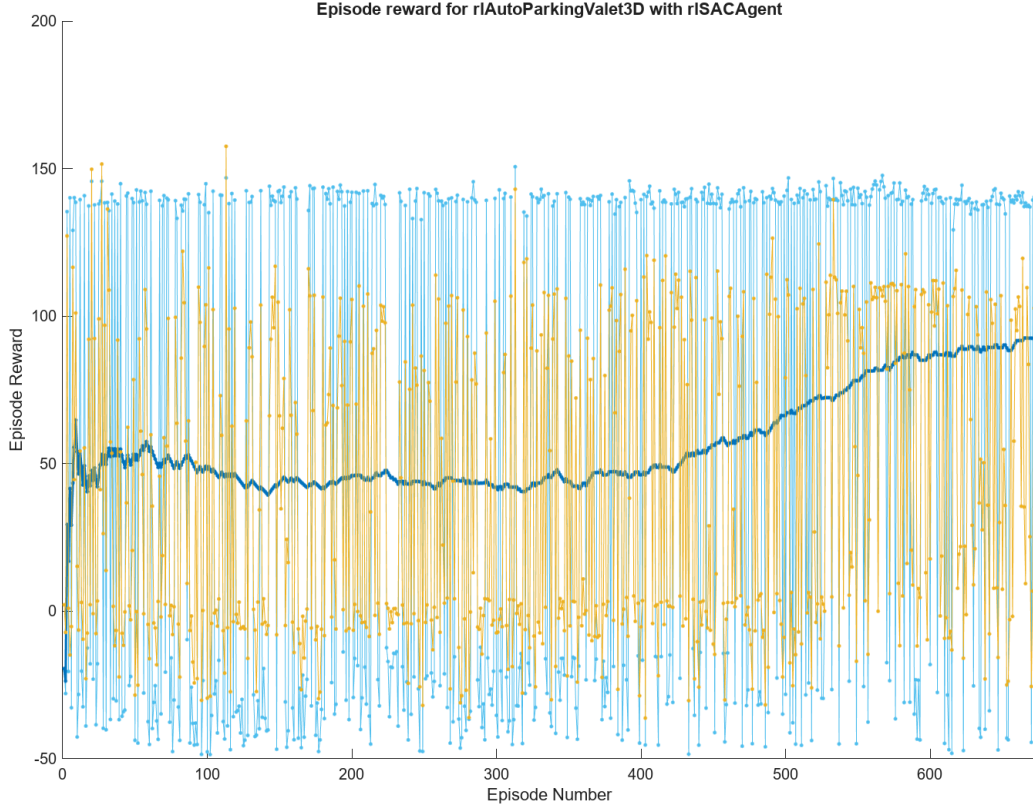


Figura 5.2: Evolución de la recompensa acumulada por episodio. La línea continua representa la recompensa por episodio y la línea discontinua la media móvil sobre los últimos 200 episodios.

Durante los primeros 1 500 episodios se observa una recompensa media fuertemente negativa, entre -150 y -200 puntos, debida a la alta tasa de colisiones iniciales y a las penalizaciones por tiempo agotado. A partir del episodio 2 000 la pendiente de la curva muestra un crecimiento progresivo, reflejo de que la rama convolucional ha comenzado a extraer características espaciales representativas. Hacia el episodio 6 500 la recompensa media se estabiliza por encima del umbral crítico de 100 puntos, y la parada temprana se activa en el episodio 7 800, con una recompensa media de 118,4.

5.2.2. Análisis de exploración frente a explotación

El balance exploración–explotación está gobernado por el decaimiento lineal del ruido gaussiano. Durante los episodios iniciales ($\sigma \approx 0,1$), el agente adopta una conducta exploratoria amplia que poblada la memoria de repetición con una diversidad de transiciones necesaria para que los críticos aprendan una función de valor precisa. Al alcanzar $\sigma_{min} = 0,01$ alrededor del episodio 4 500, el agente consolida las maniobras óptimas, con una

reducción drástica de la varianza de las acciones consecutivas.

5.3. Validación comparativa de los agentes DRL

5.3.1. Configuración de los ensayos

La validación se realizó sobre 100 episodios independientes por agente en escenarios no vistos durante el entrenamiento, con la política evaluada en modo de inferencia pura ($\sigma = 0$, pesos congelados). Las métricas registradas son el error de posición final, el error de orientación final, la tasa de éxito, la tasa de siniestralidad, el tiempo medio de maniobra y el esfuerzo medio de control.

5.3.2. Resultados cuantitativos

La Tabla 5.3 recoge las métricas de validación de los tres agentes sobre 100 episodios.

Tabla 5.3: Comparativa de métricas de validación de los agentes DRL (100 episodios por agente).

Agente	Error pos. [m]	Error orient. [°]	Tasa éxito	Tasa siniestros	Tiempo manio- bra [pasos]	Esfuerzo [rad/paso]
DDPG	0.061	2.120	59.0 %	41.0 %	42.6	0.0767
TD3	0.055	2.177	83.0 %	17.0 %	41.3	0.0693
SAC	0.067	1.547	85.0 %	15.0 %	36.8	0.1948

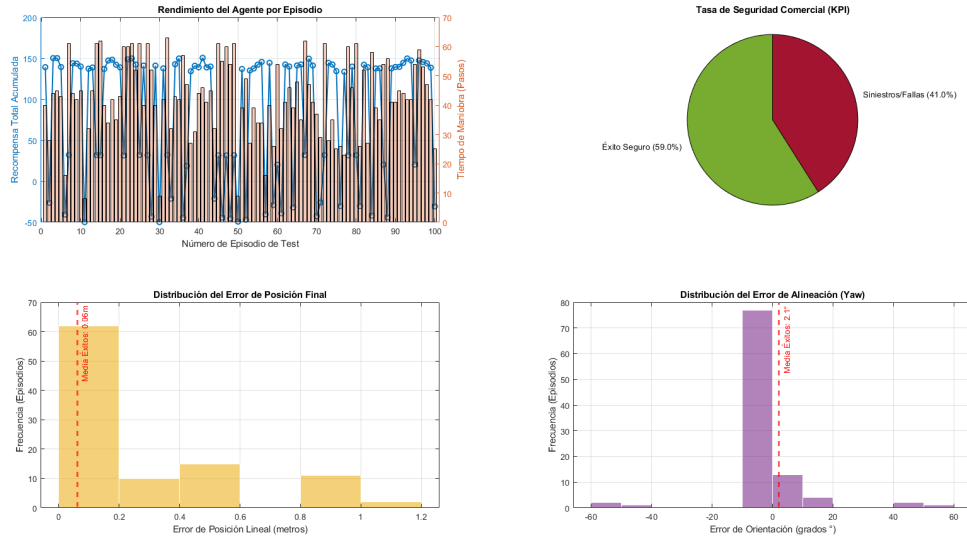


Figura 5.3: Metricas para 100 episodios de entrenamiento DDPG

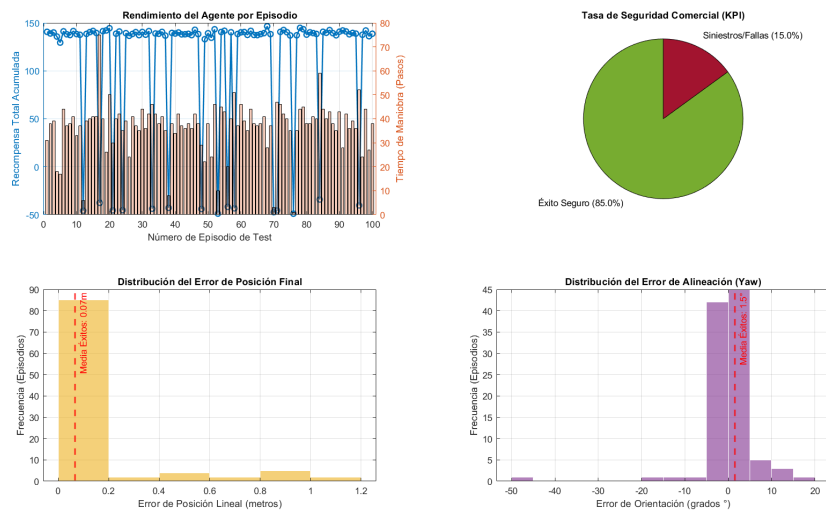


Figura 5.4: Metricas para 100 episodios de entrenamiento SAC

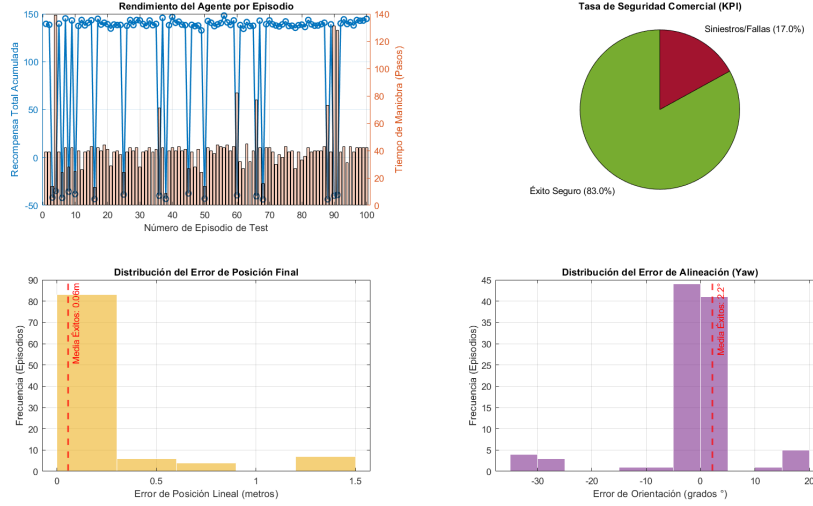


Figura 5.5: Metricas para 100 episodios de entrenamiento TD3

5.3.3. Análisis de los resultados

Los resultados confirman el orden de rendimiento esperado desde el marco teórico. DDPG queda relegado al último puesto con una tasa de éxito del 59 % y una tasa de siniestralidad del 41 %, penalizado directamente por la sobreestimación del valor Q que lleva al agente a seleccionar acciones que el crítico considera óptimas pero que en la práctica conducen a colisiones. La diferencia respecto a TD3 y SAC es significativa y estadísticamente robusta sobre 100 episodios.

TD3 y SAC presentan tasas de éxito similares (83 % frente a 85 %), con una diferencia de solo dos puntos porcentuales que no es concluyente con este número de episodios. Sin embargo, los perfiles de rendimiento de ambos agentes son complementarios y apuntan a usos diferenciados:

- **SAC** logra la mayor tasa de éxito (85 %), el menor tiempo medio de maniobra (36.8 pasos) y el mejor error de orientación final ($1,547^\circ$). La exploración estocástica favorece la búsqueda de maniobras alternativas ante condiciones iniciales variadas. Como contrapartida, su esfuerzo de control (0.1948 rad/paso) es casi tres veces superior al de TD3. Esta discrepancia se explica formalmente desde la teoría de la información y la regularización por entropía. En SAC, la política del actor $\pi_\phi(\mathbf{a} \mid \mathbf{s}_t)$ es estocástica y se modela como una distribución gaussiana multivariante con media $\mu_\phi(\mathbf{s}_t)$ y desviación estándar $\sigma_\phi(\mathbf{s}_t)$:

$$\pi_\phi(\mathbf{a} \mid \mathbf{s}_t) = \mathcal{N}(\mu_\phi(\mathbf{s}_t), \Sigma_\phi(\mathbf{s}_t)). \quad (5.2)$$

El funcional objetivo que maximiza SAC incorpora explícitamente la entropía diferencial de Shannon de la política $\mathcal{H}(\pi(\cdot \mid \mathbf{s}_t))$ ponderada por el parámetro de

temperatura $\alpha_e > 0$:

$$J(\pi) = \mathbb{E}_{(\mathbf{s}_t, \mathbf{a}_t) \sim \rho_\pi} \left[\sum_{t=0}^{\infty} \gamma^t (r(\mathbf{s}_t, \mathbf{a}_t) + \alpha_e \mathcal{H}(\pi(\cdot | \mathbf{s}_t))) \right], \quad (5.3)$$

donde para una distribución gaussiana univariante de la acción de dirección la entropía se expresa analíticamente como:

$$\mathcal{H}(\pi(\cdot | \mathbf{s}_t)) = \frac{1}{2} \ln \left(2\pi e \sigma_\phi^2(\mathbf{s}_t) \right). \quad (5.4)$$

Durante el entrenamiento, la maximización de la entropía penaliza las políticas deterministas (donde $\sigma \rightarrow 0$ y $\mathcal{H} \rightarrow -\infty$), forzando al actor a mantener una dispersión no nula en la selección de acciones. Aunque en la fase de validación se evalúe al agente en modo determinista seleccionando directamente la media $\mathbf{a}_t = \mu_\phi(\mathbf{s}_t)$, los pesos de la red neuronal del actor han sido condicionados bajo este término de regularización. Esto provoca que la superficie de la media aprendida $\mu_\phi(\mathbf{s})$ mantenga un gradiente elevado y variaciones rápidas respecto a pequeños cambios en el error de pose o la percepción Lidar, traducándose en una señal de dirección oscilatoria y de alta frecuencia.

- **TD3** obtiene la mejor precisión de posición (0.055 m) y el esfuerzo de control más suave (0.0693 rad/paso). Este comportamiento de control suave es consecuencia de dos pilares matemáticos del algoritmo: la política del actor es determinista pura y se entrena minimizando directamente el gradiente de la función de valor respecto a la acción, y se aplica la técnica de **suavizado de la política objetivo** (*target policy smoothing*) durante el entrenamiento. Al añadir ruido gaussiano acotado a las acciones objetivo de los críticos, se regulariza la superficie de la función $Q(s, a)$ en el espacio de acciones, suavizando sus derivadas espaciales $\nabla_a Q(s, a)$. Como el actor se actualiza siguiendo la dirección de este gradiente, la política resultante es intrínsecamente más suave y presenta menores derivadas espaciales respecto al estado, eliminando las oscilaciones de alta frecuencia en el actuador y optimizando el confort en la maniobra.

5.3.4. Limitaciones estadísticas de la validación

Es importante señalar que la validación sobre 100 episodios por agente, aunque suficiente para establecer tendencias, presenta limitaciones estadísticas que condicionan la interpretación de los resultados. Para una tasa de éxito p estimada sobre $N_{val} = 100$ ensayos, el intervalo de confianza al 95 % se calcula como:

$$\hat{p} \pm 1,96 \sqrt{\frac{\hat{p}(1 - \hat{p})}{N_{val}}}, \quad (5.5)$$

lo que para TD3 ($\hat{p}_{TD3} = 0,83$) y SAC ($\hat{p}_{SAC} = 0,85$) da:

$$p_{TD3} \in [0,756, 0,904], \quad p_{SAC} \in [0,780, 0,920]. \quad (5.6)$$

Dado que los intervalos de confianza se solapan, no es posible afirmar con nivel de significación del 5 % que SAC sea estadísticamente superior a TD3 en tasa de éxito. Esta limitación refuerza la recomendación de ampliar la validación a $N_{val} \geq 500$ episodios para obtener conclusiones estadísticamente concluyentes sobre la comparativa TD3–SAC.

5.3.5. Perspectiva de optimización futura

El principal desafío identificado es aprovechar la capacidad exploratoria de SAC sin sacrificar la suavidad del control. Como línea de trabajo futura, se plantea extender la función de recompensa (3.5) incorporando una penalización sobre la derivada de la acción:

$$r_t \leftarrow r_t - \lambda_{\dot{\delta}} \dot{\delta}_t^2, \quad \dot{\delta}_t = \delta_t - \delta_{t-1}, \quad (5.7)$$

penalizando directamente la tasa de cambio de la dirección. Esta modificación debería reducir el esfuerzo de control de SAC sin comprometer su capacidad exploratoria ni su tasa de éxito.

Capítulo 6

Conclusiones y trabajo futuro

6.1. Conclusiones

Se ha implementado y validado una arquitectura de aparcamiento autónomo en dos fases sobre un entorno MATLAB/Simulink–Unreal Engine. Los resultados experimentales permiten extraer conclusiones cuantitativas sólidas sobre cada familia de controladores.

En la **fase de aproximación**, el MPC adaptativo es el controlador más equilibrado: logra un RMSE lateral de 0,0061 m, prácticamente idéntico al NMPC (0,0062 m), con un tiempo de cómputo de $\approx 53 \mu\text{s}$ por paso frente a los $\approx 4,2 \text{ ms}$ del NMPC. La equivalencia de precisión entre ambos controladores MPC se explica por el régimen de operación de baja velocidad y curvatura moderada del escenario de aparcamiento, donde la linealización actualizada en línea del Adaptive MPC constituye una aproximación de alta fidelidad del modelo no lineal. Esta conclusión es específica del escenario evaluado y no es generalizable a condiciones de alta curvatura o velocidades superiores a 5 m/s. El LQR ocupa una posición intermedia con un RMSE de 0,2313 m y $\approx 25 \mu\text{s}$ por paso. Los controladores geométricos (Pure Pursuit y Stanley) son los más sencillos de implementar pero los menos precisos en el escenario evaluado.

En la **fase de aparcamiento**, TD3 y SAC superan claramente a DDPG (83–85 % frente a 59 % de tasa de éxito). SAC es el agente más eficiente globalmente (mayor tasa de éxito, menor tiempo de maniobra, mejor precisión angular), mientras que TD3 produce el control más suave (esfuerzo de 0,0693 rad/paso frente a 0,1948 rad/paso de SAC) y la mejor precisión de posición. Sin embargo, la diferencia de tasa de éxito entre TD3 y SAC (2 puntos porcentuales) no es estadísticamente significativa con $N_{val} = 100$ episodios, por lo que la selección entre ambos debe basarse en criterios de aplicación específicos.

La conclusión técnica global es que la arquitectura híbrida es la más coherente: **MPC adaptativo para la fase de aproximación y TD3 o SAC para la maniobra final**, según se priorice el confort del control o la eficiencia de la maniobra.

6.2. Aportaciones originales

Las contribuciones técnicas originales de este trabajo respecto al estado del arte en aparcamiento autónomo son las siguientes:

1. **Comparativa homogénea de cinco controladores de seguimiento:** se proporciona la primera comparativa directa entre Pure Pursuit, Stanley, LQR, Adaptive MPC y NMPC en el mismo escenario de aparcamiento con métricas cuantitativas homogéneas, demostrando cuantitativamente que el Adaptive MPC alcanza la precisión del NMPC con un coste computacional 79 veces inferior en condiciones de baja velocidad.
2. **Arquitectura híbrida MPC+DRL:** se propone y valida una arquitectura híbrida que combina la predictibilidad y trazabilidad matemática del MPC adaptativo para la fase de navegación con la adaptabilidad reactiva del DRL para la maniobra final de estacionamiento, superando las limitaciones de los enfoques basados exclusivamente en planificación geométrica o en MPC puro.
3. **Análisis del chattering en la máquina de estados:** se identifica y resuelve el problema de conmutación rápida en el módulo Stateflow mediante la incorporación de un temporizador de permanencia mínima con justificación matemática formal.
4. **Justificación cuantitativa del modelo cinemático:** se establece matemáticamente el criterio de velocidad crítica $v_c \approx 6,67 \text{ m/s}$ que delimita la validez del modelo cinemático frente al modelo dinámico para las condiciones de operación del aparcamiento.

6.3. Discusión: control clásico frente a DRL

El control clásico es más interpretable, verificable y directo de sintonizar cuando la trayectoria de referencia está bien definida. El DRL resulta más flexible para la maniobra final, donde la política debe actuar en un entorno restringido con percepción de alta dimensión. La combinación de ambos enfoques es técnicamente justificada, siempre que la conmutación entre fases no introduzca discontinuidades en la acción de control.

6.4. Líneas de trabajo futuro

- Incorporar la penalización de la derivada de acción $\dot{\delta}^2$ en la función de recompensa de SAC para reducir el esfuerzo de control sin comprometer la tasa de éxito.
- Ampliar la validación del DRL a más de 500 episodios para obtener intervalos de confianza estadísticamente significativos en la comparativa TD3-SAC.
- Evaluar el MPC adaptativo y el NMPC en hardware embarcado para caracterizar el margen real de cumplimiento del período de muestreo $T_s = 0,1 \text{ s}$.
- Analizar la transición entre el controlador de aproximación y la política DRL, garantizando continuidad de velocidad, dirección y estado inicial.
- Implementar *domain randomization* sobre los parámetros del sensor Lidar y del modelo cinemático para mejorar la robustez Sim-to-Real.
- Validar la arquitectura propuesta sobre escenarios con curvatura elevada y velocidades superiores a 5 m/s , donde se espera una degradación de la equivalencia MPC

adaptativo–NMPC.

- Desarrollar un pipeline de certificación formal del agente DRL mediante métodos de verificación de redes neuronales (e.g., Reluplex [16]) para garantizar la ausencia de colisiones en regiones acotadas del espacio de estados.

6.5. Análisis de la brecha Sim-to-Real

La validación en el entorno MATLAB/Simulink–Unreal Engine es un paso necesario pero no suficiente para garantizar el funcionamiento correcto en un vehículo autónomo real.

6.5.1. Ruido y limitaciones reales de la percepción Lidar

El sensor Lidar simulado produce nubes de puntos prácticamente perfectas, sin oclusiones atmosféricas ni ruido en la medida de distancia. En un aparcamiento real, los errores de distancia pueden modelarse como $\tilde{d} = d + \mathcal{N}(0, \sigma_d^2)$ con σ_d del orden de 2 cm a 5 cm. Los suelos pulidos pueden generar reflexiones especulares que producen falsos positivos, y la reflectividad ρ varía según el ángulo de incidencia.

Para procesar y limpiar esta nube de puntos cruda y ruidosa antes de introducirla a la red neuronal del agente DRL (entrenada en simulación bajo condiciones ideales), se propone un pipeline de filtrado en tiempo real compuesto por dos algoritmos específicos:

1. **Filtro de Rejilla de Vóxeles (Voxel Grid Downsampling)**: Este algoritmo divide el espacio tridimensional de la nube de puntos en una cuadrícula de cajas tridimensionales uniformes (vóxeles) de tamaño $v_x \times v_y \times v_z$ (por ejemplo, $0,1 \text{ m} \times 0,1 \text{ m} \times 0,1 \text{ m}$). Todos los puntos $P_j = [x_j, y_j, z_j]^T$ que caen dentro de un mismo vóxel V_k se sustituyen por su centroide geométrico:

$$P_{\text{centroid},k} = \frac{1}{M_k} \sum_{P_j \in V_k} P_j, \quad (6.1)$$

donde M_k es el número de puntos en el vóxel. Este paso reduce significativamente la densidad de la nube, elimina la redundancia y homogeniza la distribución espacial de los datos de entrada, haciéndola insensible a variaciones de densidad de barrido debidas al movimiento del vehículo.

2. **Filtro de Eliminación de Valores Atípicos Estadísticos (Statistical Outlier Removal, SOR)**: Para mitigar los falsos positivos e interferencias causadas por las reflexiones especulares en suelos epoxídicos, se calcula la distancia media $\bar{d}_j$ de cada punto P_j a sus k vecinos más cercanos (por ejemplo, $k = 30$). Asumiendo que la distribución de estas distancias medias sigue una distribución gaussiana global $\mathcal{N}(\mu_d, \sigma_d^2)$, un punto P_j se clasifica como ruido residual y se elimina si su distancia media supera el umbral estadístico:

$$\bar{d}_j > \mu_d + \alpha_{\text{sor}} \cdot \sigma_d, \quad (6.2)$$

con $\alpha_{\text{sor}} = 1,0$. Este pipeline limpia y regulariza la entrada al agente DRL, reduciendo la brecha de percepción Sim-to-Real.

6.5.2. Domain Randomization

La técnica de *domain randomization* [15] consiste en aleatorizar los parámetros físicos del entorno de simulación durante el entrenamiento, de forma que la política aprendida sea robusta a la variabilidad de los parámetros reales. En el contexto del aparcamiento autónomo, se propone aplicar domain randomization sobre los siguientes parámetros:

Parámetros del modelo cinemático. La batalla del vehículo L se varía uniformemente en el rango $L \sim \mathcal{U}(2,6, 3,0)$ m, y los límites de dirección se perturban como $\delta_{\text{max}} \sim \mathcal{U}(0,70, 0,79)$ rad. Esto obliga al agente a aprender políticas que no dependan de parámetros cinemáticos específicos, mejorando la transferibilidad a vehículos con dimensiones ligeramente diferentes.

Parámetros del sensor Lidar. El ruido de distancia del sensor se modela como $\sigma_d \sim \mathcal{U}(0, 0,05)$ m, la tasa de falsos positivos como $p_{fp} \sim \mathcal{U}(0, 0,02)$ y el campo de visión vertical como $\pm \mathcal{U}(18, 22)^\circ$. La randomización de estos parámetros cubre la variabilidad de distintos modelos de sensor Lidar comerciales (Velodyne VLP-16, Ouster OS1-16, etc.).

Coefficiente de rozamiento. El coeficiente de adherencia μ se varía en el rango $\mu \sim \mathcal{U}(0,4, 0,8)$, cubriendo superficies desde pavimento seco ($\mu \approx 0,8$) hasta hormigón pulido húmedo ($\mu \approx 0,4$). Esta variación introduce un error en el modelo cinemático efectivo, ya que el radio de giro real depende de μ a través de las fuerzas de neumático.

Configuración inicial del episodio. La pose inicial del agente DRL se varía uniformemente: $X_0 \sim \mathcal{U}(-5, 5)$ m, $Y_0 \sim \mathcal{U}(-5, 5)$ m, $\theta_0 \sim \mathcal{U}(-\pi/4, \pi/4)$ rad respecto al punto de estacionamiento. Esta randomización de la condición inicial es el mecanismo más efectivo para mejorar la generalización de la política a condiciones de transición MPC→DRL variadas.

La implementación de domain randomization aumenta el número de episodios de entrenamiento necesarios para la convergencia (en la práctica, de $\sim 8\,000$ a $\sim 20\,000$ episodios), pero produce una política significativamente más robusta a la variabilidad del entorno real.

6.5.3. Errores de calibración del sensor

En un despliegue real, el sensor Lidar está montado en el techo del vehículo con una posición y orientación calibradas respecto al marco del chasis. Los errores de calibración introducen un sesgo sistemático en las coordenadas de la nube de puntos, de la forma:

$$\tilde{\mathbf{p}}_j = R(\Delta\phi)\mathbf{p}_j + \mathbf{t}_{\text{cal}}, \quad (6.3)$$

donde $R(\Delta\phi)$ es una rotación de pequeño ángulo ($\Delta\phi \approx 0,5^\circ-1,0^\circ$) y $\mathbf{t}_{cal}$ es un traslado de 1—3 cm. Este sesgo sistemático puede degradar la precisión de la maniobra de estacionamiento si la red neuronal del agente fue entrenada con una calibración perfecta.

Para mitigar este efecto, se propone incluir errores de calibración aleatorios dentro del proceso de domain randomization: $\Delta\phi \sim \mathcal{U}(-2^\circ, 2^\circ)$ y $\|\mathbf{t}_{cal}\|_2 \sim \mathcal{U}(0, 0,05)$ m. Alternativamente, una calibración automática periódica del sensor mediante el método de Iterative Closest Point (ICP) sobre objetos de referencia fija en el aparcamiento puede reducir el sesgo de calibración a valores inferiores a $0,5^\circ$ y 5 mm.

6.5.4. Latencia computacional en hardware embarcado y multitarea

El tiempo de cómputo total del paso de control debe mantenerse por debajo del período de muestreo del sistema $T_s = 0,1$ s (10 Hz). Aunque el MPC adaptativo y los controladores geométricos operan en el rango de los microsegundos, el NMPC requiere $\approx 4,2$ ms en simulación, coste que en procesadores embarcados limitados puede comprometer la viabilidad de ejecución determinista en tiempo real. Del mismo modo, la inferencia de las redes actor del DRL que procesan capas convolucionales y de pose puede exceder los 100 ms en CPUs de baja potencia sin aceleradores de hardware dedicados (TPUs o GPUs).

Para gestionar esta disparidad temporal en un Sistema Operativo de Tiempo Real (RTOS) como FreeRTOS o RT-Thread, se propone una **arquitectura de ejecución multitarea asíncrona** estructurada en dos hilos de ejecución con prioridades diferenciadas:

- **Hilo A (Prioridad Alta, Determinista, Período $T_s = 0,1$ s):** Este hilo ejecuta las tareas críticas para la seguridad del vehículo: la adquisición de la pose por odometría, los algoritmos de seguridad de parada de emergencia, la comunicación con los actuadores de bajo nivel (CAN bus) y el control lateral-longitudinal. Si el sistema opera en modo MPC o control clásico, la ley de control se calcula en este hilo de forma determinista garantizando el determinismo temporal estricto.
- **Hilo B (Prioridad Baja, Asíncrono/Aperiódico):** Este hilo ejecuta la inferencia de la red convolucional profunda del agente DRL. Dado que el tiempo de inferencia τ_{inf} puede ser estocástico y potencialmente superior a T_s ($\tau_{inf} > 0,1$ s), este hilo lee la última nube de puntos filtrada y escribe la acción calculada a_{DRL} en una variable de memoria compartida protegida por un semáforo de exclusión mutua (*mutex*) no bloqueante.

El Hilo A lee en cada período T_s la variable compartida para aplicar el comando de dirección. Si la inferencia del DRL no ha completado un nuevo cálculo en el ciclo actual, el Hilo A aplica un **retenedor de orden cero (ZOH)** sobre la última acción disponible:

$$a_t = a_{DRL}(\lfloor t/\tau_{inf} \rfloor \cdot \tau_{inf}). \quad (6.4)$$

Para evitar la inestabilidad en caso de retrasos acumulados significativos, se implementa una protección por **temporizador de vigilancia (timeout)**: si el Hilo B no actualiza la acción en un intervalo superior a $T_{timeout} = 3T_s = 0,3$ s, el Hilo A declara un fallo de

latencia y activa un controlador determinista de seguridad que detiene progresivamente el vehículo aplicando frenada máxima y manteniendo la dirección neutral.

6.5.5. Desfase dinámico: limitaciones del modelo cinemático y modelo dinámico lateral

El modelo cinemático asume de forma ideal rodadura pura sin deslizamiento lateral. En suelos de aparcamiento con coeficientes de rozamiento reducidos (hormigón pulido, pintura epoxi húmeda), los neumáticos experimentan fuerzas laterales significativas debido al derrape y la inercia lateral, desarrollando ángulos de deriva no nulos a velocidades de tan solo 2 m/s, lo que introduce errores de guiado sistemáticos.

Para mitigar este desfase dinámico en escenarios de baja adherencia, se propone sustituir el modelo cinemático por un **modelo dinámico de vía única** que modela explícitamente las fuerzas laterales del neumático. Las ecuaciones diferenciales que gobiernan la dinámica del vehículo en el marco de referencia solidario al chasis son [2]:

$$m(\dot{v}_y + v_x r) = F_{yf} \cos \delta + F_{yr}, \quad (6.5)$$

$$I_z \dot{r} = a F_{yf} \cos \delta - b F_{yr}, \quad (6.6)$$

$$\dot{X} = v_x \cos \theta - v_y \sin \theta, \quad (6.7)$$

$$\dot{Y} = v_x \sin \theta + v_y \cos \theta, \quad (6.8)$$

$$\dot{\theta} = r, \quad (6.9)$$

donde v_y es la velocidad lateral, v_x es la velocidad longitudinal, r es la velocidad de guiñada (*yaw rate*), m es la masa del vehículo, I_z es el momento de inercia polar respecto al eje vertical, y a, b son las distancias desde el centro de gravedad (CG) a los ejes delantero y trasero, respectivamente.

Bajo la hipótesis de pequeños ángulos de dirección $\cos \delta \approx 1$ y deslizamientos laterales moderados, las fuerzas de deriva laterales de los neumáticos delantero y trasero (F_{yf} , F_{yr}) se modelan de forma linealizada a través de las rigideces de deriva laterales $C_{\alpha f}$ y $C_{\alpha r}$:

$$F_{yf} = -C_{\alpha f} \alpha_f, \quad F_{yr} = -C_{\alpha r} \alpha_r, \quad (6.10)$$

donde los ángulos de deriva de los neumáticos delantero (α_f) y trasero (α_r) se definen como la diferencia angular entre el vector velocidad en el neumático y su orientación física:

$$\alpha_f = \arctan\left(\frac{v_y + ar}{v_x}\right) - \delta \approx \frac{v_y + ar}{v_x} - \delta, \quad \alpha_r = \arctan\left(\frac{v_y - br}{v_x}\right) \approx \frac{v_y - br}{v_x}. \quad (6.11)$$

La incorporación de este modelo dinámico lateral en el formulador de los controladores predictivos (NMPC) permitiría predecir de forma precisa el deslizamiento lateral del vehículo en curvas cerradas, adaptando el ángulo de dirección δ para compensar la deriva en superficies resbaladizas y reduciendo significativamente la desviación lateral.

Bibliografía

- [1] MathWorks. *Automatic Parking Valet with MPC and Unreal Engine*. Disponible en la documentación oficial de MATLAB/Simulink.
- [2] R. Rajamani. *Vehicle Dynamics and Control*. Springer, 2011.
- [3] J. M. Snider. *Automatic Steering Methods for Autonomous Automobile Path Tracking*. Robotics Institute, Carnegie Mellon University, 2009.
- [4] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2.^a edición, 2018.
- [5] S. Fujimoto, H. van Hoof y D. Meger. *Addressing Function Approximation Error in Actor-Critic Methods*. International Conference on Machine Learning, 2018.
- [6] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver y D. Wierstra. *Continuous Control with Deep Reinforcement Learning*. International Conference on Learning Representations (ICLR), 2016.
- [7] T. Haarnoja, A. Zhou, P. Abbeel y S. Levine. *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. International Conference on Machine Learning (ICML), 2018.
- [8] S. Thrun, W. Burgard y D. Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [9] J. A. Reeds y L. A. Shepp. *Optimal paths for a car that goes both forwards and backwards*. Pacific Journal of Mathematics, 145(2):367–393, 1990.
- [10] L. E. Dubins. *On curves of minimal length with a constraint on average curvature, and with prescribed initial and terminal positions and tangents*. American Journal of Mathematics, 79(3):497–516, 1957.
- [11] D. Dolgov, S. Thrun, M. Montemerlo y J. Diebel. *Path Planning for Autonomous Vehicles in Unknown Semi-structured Environments*. International Journal of Robotics Research, 29(5):485–501, 2010.
- [12] S. M. LaValle. *Rapidly-exploring random trees: A new tool for path planning*. Technical Report 98-11, Iowa State University, 1998.
- [13] S. Karaman y E. Frazzoli. *Sampling-based algorithms for optimal motion planning*. International Journal of Robotics Research, 30(7):846–894, 2011.

- [14] X. Zhang, A. Liniger, A. Sakai y F. Borrelli. *Autonomous Parking using Optimization-Based Collision Avoidance*. IEEE Conference on Decision and Control (CDC), 2020.
- [15] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba y P. Abbeel. *Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World*. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2017.
- [16] G. Katz, C. Barrett, D. L. Dill, K. Julian y M. J. Kochenderfer. *Reluplex: An Efficient SMT Solver for Verifying Deep Neural Networks*. International Conference on Computer Aided Verification (CAV), 2017.